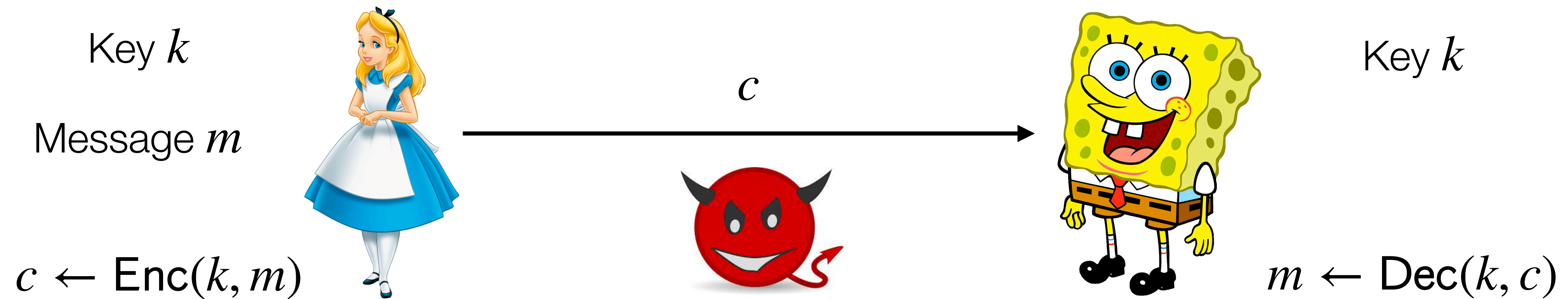


CS 350S: Privacy-Preserving Systems

Cryptography basics

Recap: Symmetric-key encryption



Confidentiality: Attacker who sees c “learns nothing” about (fixed-size) message

Recap: Symmetric-key encryption

One-time pad

- $\text{Enc}(k, m) : k \oplus m$
- $\text{Dec}(k, c) : k \oplus c$

Drawbacks:

- Key is as long as the message
- Only use once

Recap: Symmetric-key encryption

We can compress the one-time pad key with a pseudorandom generator (PRG)
PRG (informal): stretch a short “seed” into a long, random-looking string

Stream cipher:

- $\text{Enc}(k, m) : \text{PRG}(k) \oplus m$
- $\text{Dec}(k, c) : \text{PRG}(k) \oplus c$

Recap: Symmetric-key encryption

Block ciphers make it possible to encrypt multiple messages with the same key

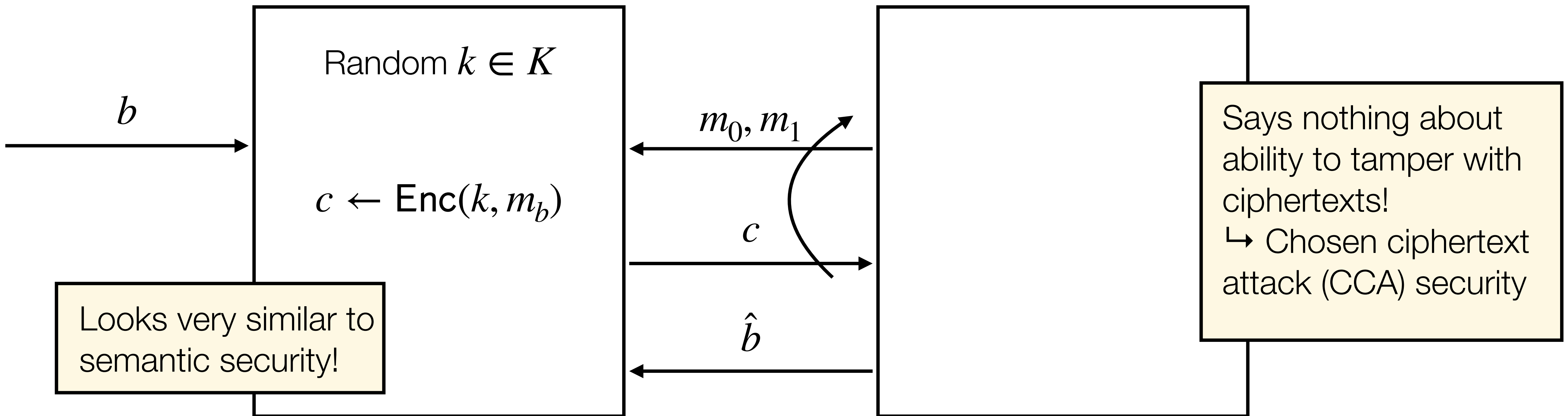
- use a fresh nonce for each key

Pseudorandom permutation (PRP): indistinguishable from a random-permutation

Recap: Chosen plaintext attack (CPA) security

Challenger

Adversary

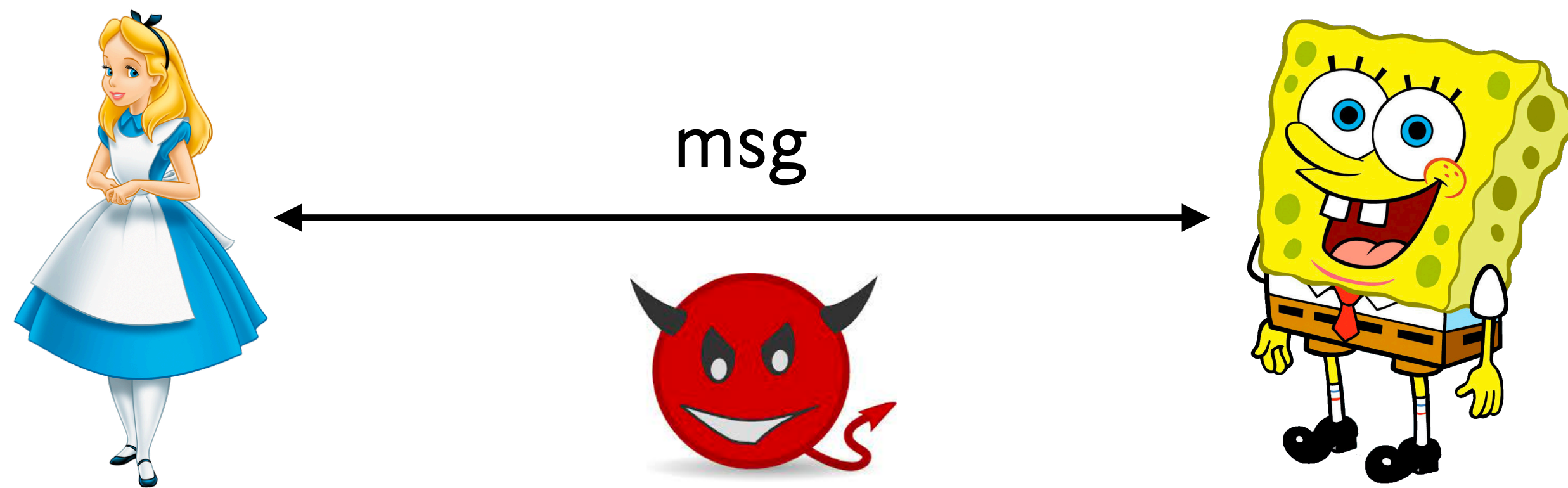


An encryption scheme is CPA-secure if the adversary's probability of guessing $\hat{b} = b$ is "very close" to $1/2$

Today: Outline

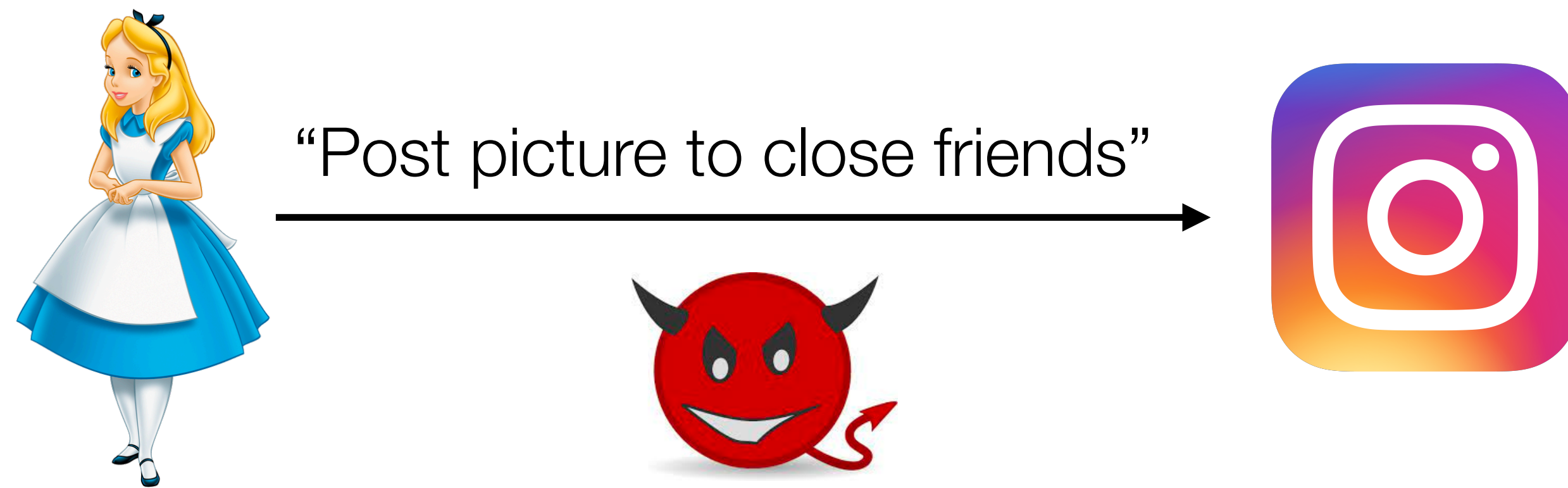
1. Symmetric-key cryptography
 - Message integrity
2. Public-key cryptography
3. Web public-key infrastructure
4. Authenticated data structures

Message integrity



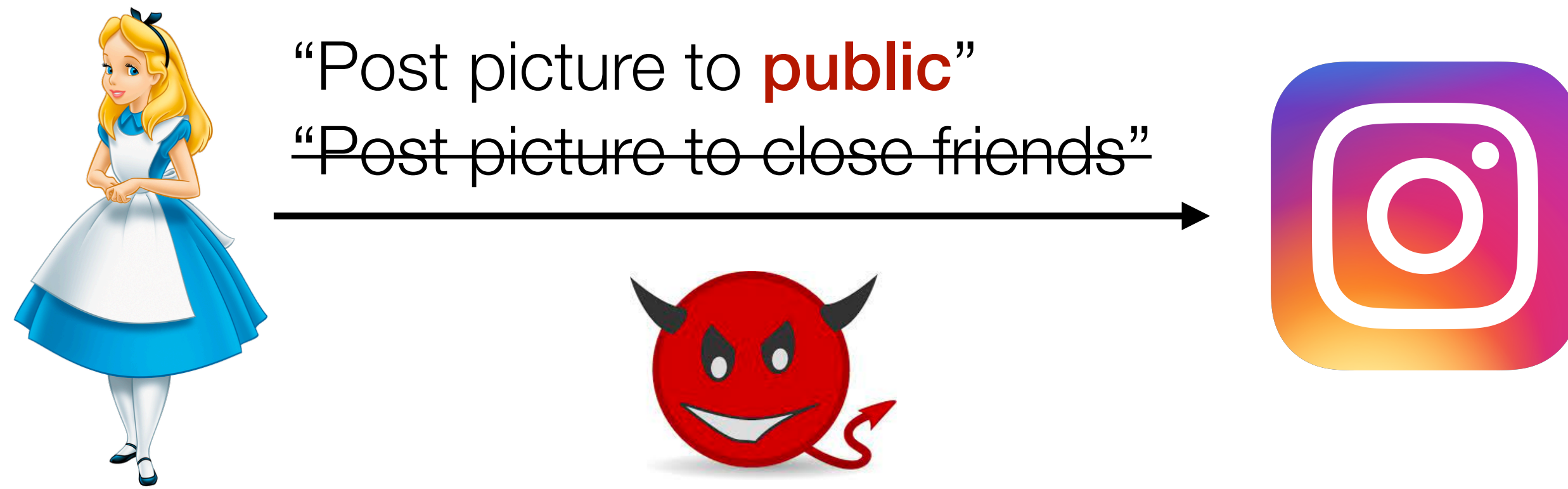
Integrity: Alice and Bob want to ensure that the attacker has not tampered with the message

Confidentiality and integrity go together



Depending on application, attacker can change a ciphertext and cause application-level damage

Confidentiality and integrity go together



Depending on application, attacker can change a ciphertext and cause application-level damage

Collision-resistant hash functions

Input space $\{0,1\}^*$

Output space $\{0,1\}^{256}$

Hash function $H : \{0,1\}^* \rightarrow \{0,1\}^{256}$

A hash function H is collision-resistant if no “efficient” adversary can find a collision, i.e.

$H(a) = H(b)$ for $a \neq b$

Note: many collisions exist, just hard to find

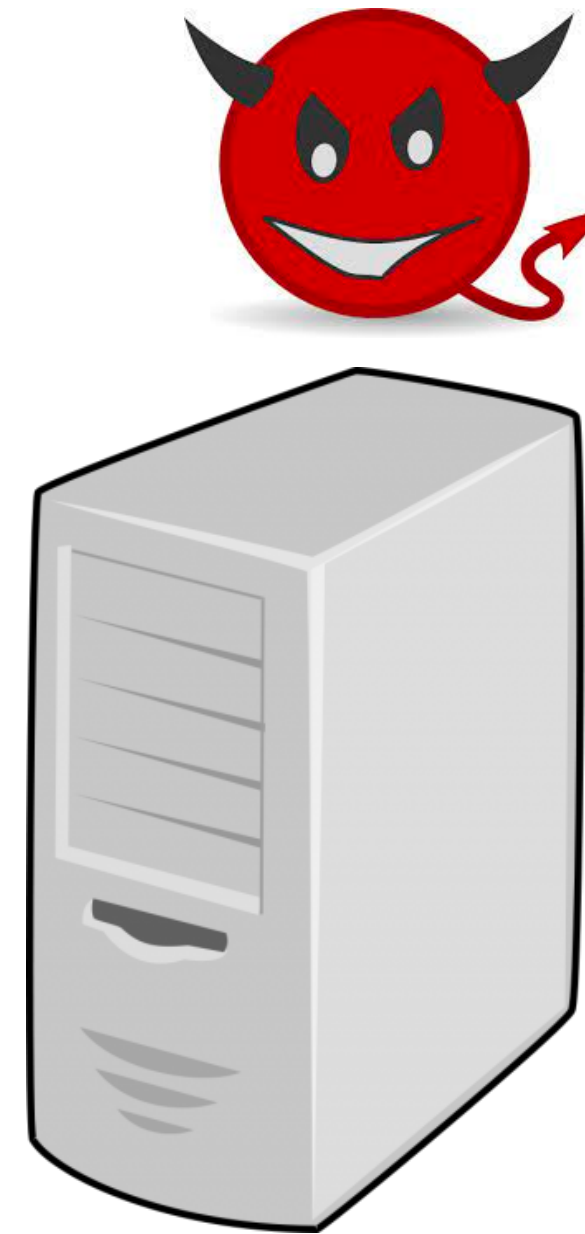
Example of collision-resistant hash function: SHA256

Application: file storage

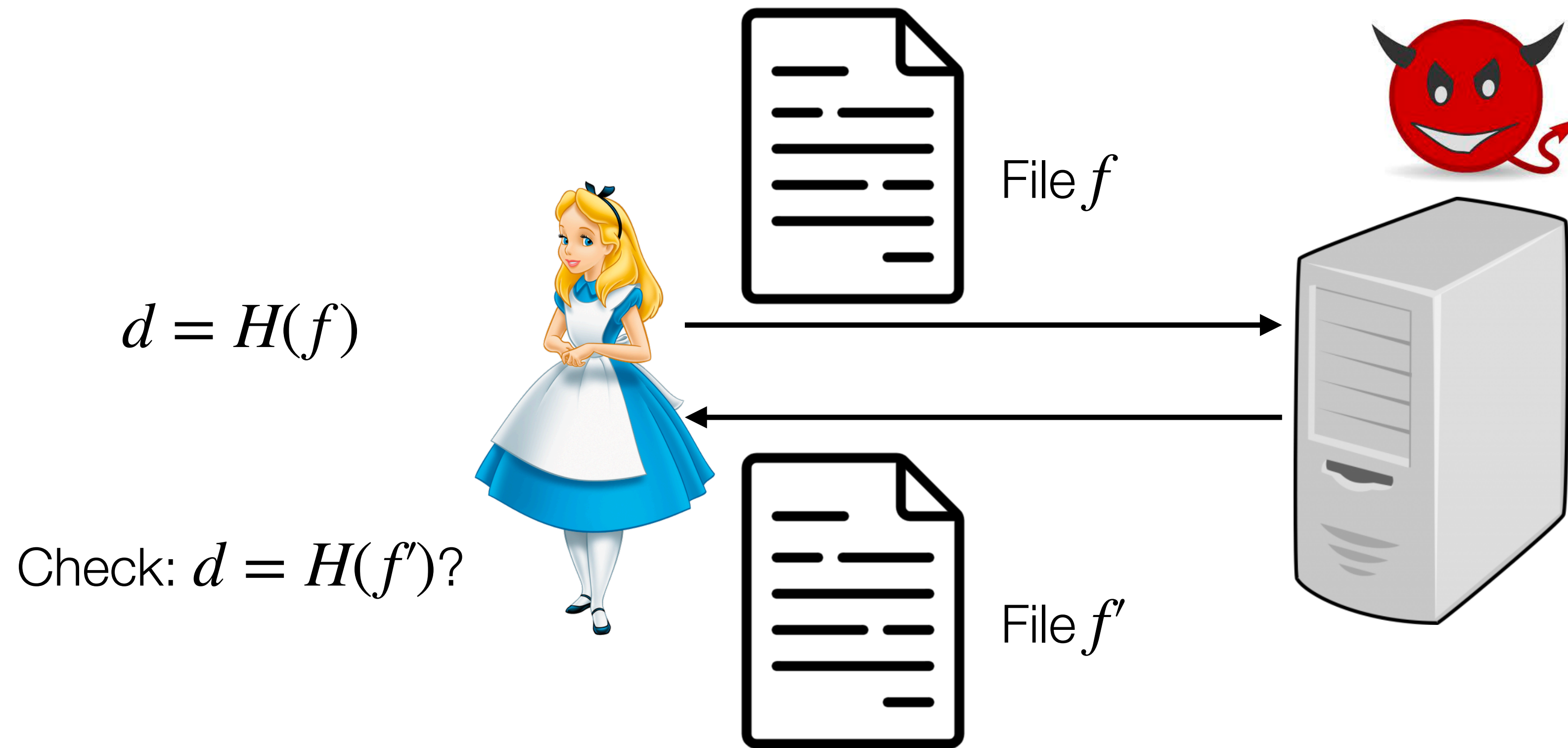
$$d = H(f)$$



File f



Application: file storage



Next: Can we avoid the client storing a hash digest for each file?

Message authentication codes (MACs)

Key space K

Message space M

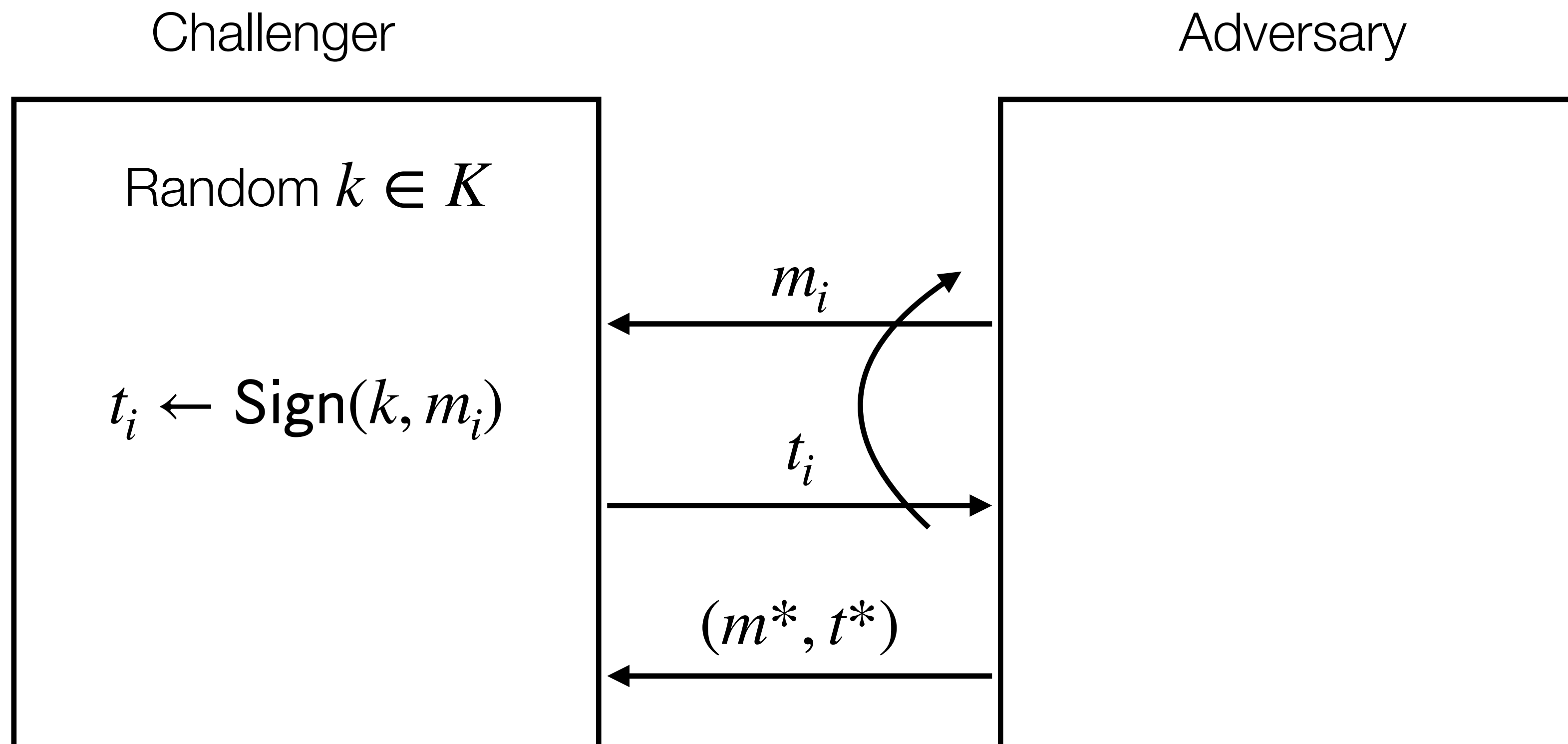
Tag space T

Message authentication codes:

- **Sign** : $K \times M \rightarrow T$
- **Verify** : $K \times M \times T \rightarrow \{0,1\}$

What is the right way to define security?

Chosen message attack security



Adversary wins if:

- $(m^*, t^*) \notin \{(m_1, t_1), (m_2, t_2), \dots\}$
- $\text{Verify}(k, m^*, t^*) = 1$ ("accept")

MACs from PRFs

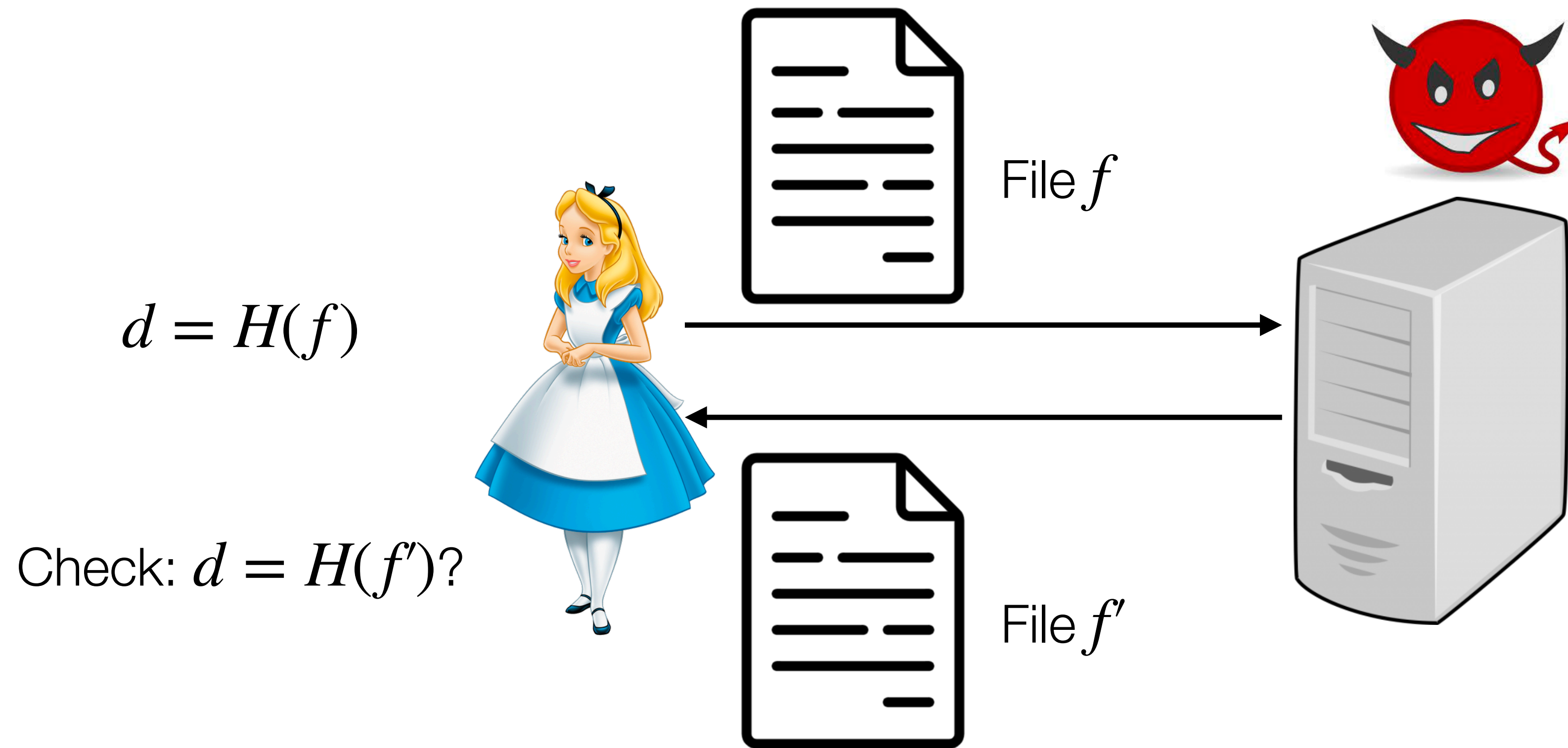
Given PRF F :

- **Sign**(k, m) : $F(k, m)$
- **Verify**(k, m, t) : Output 1 (“accept”) if $F(k, m) = t$, 0 (“reject”) otherwise

Note: this MAC construction is deterministic

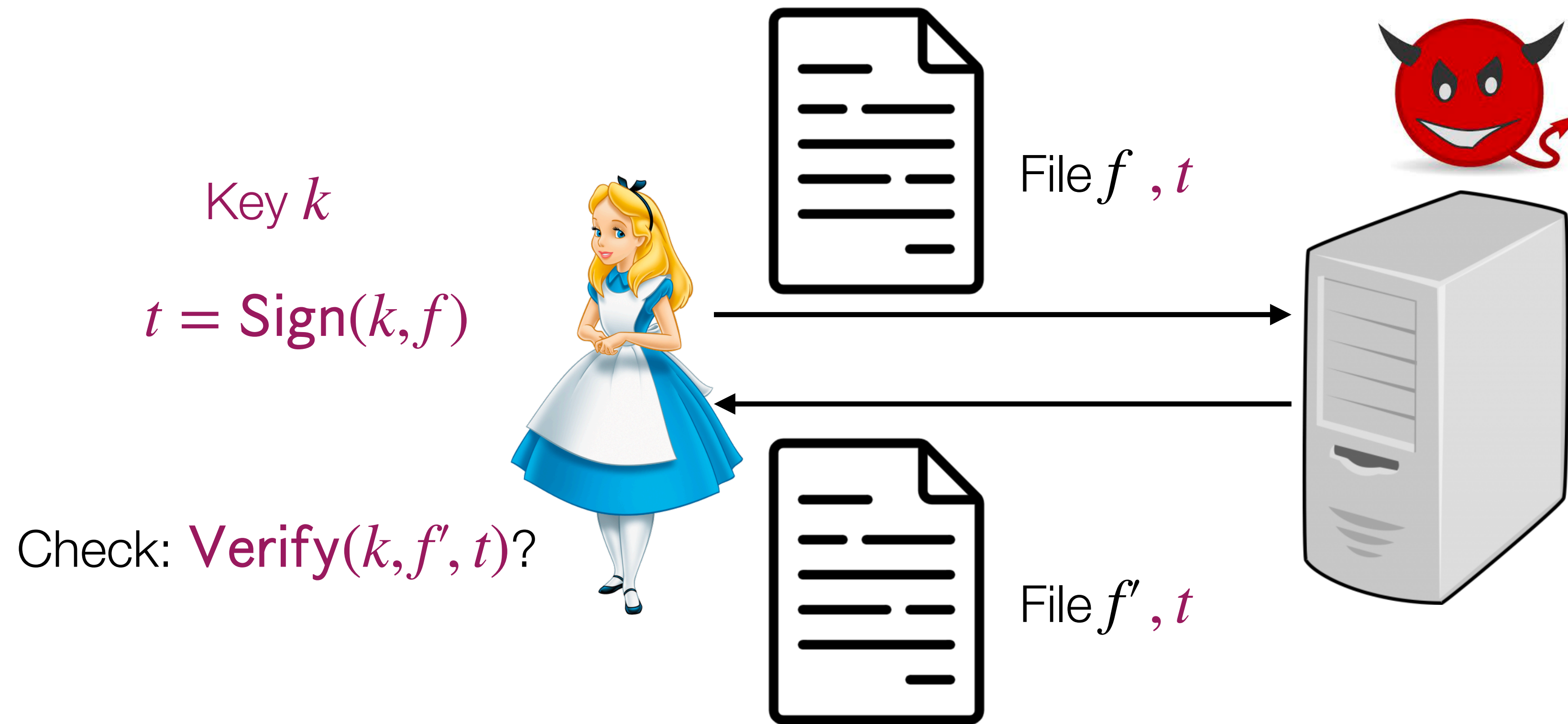
Another example of MAC: HMAC (from SHA256)

Revisiting file storage



Can we avoid the client storing a hash digest for each file?

Revisiting file storage



Alice can outsource many files and only store 1 key

Q: other applications for MACs?

Confidentiality + Integrity: Authenticated encryption

Encryption key k_E . MAC key = k_I

Option 1: (SSL)



Option 2: (IPsec, TLS 1.3)

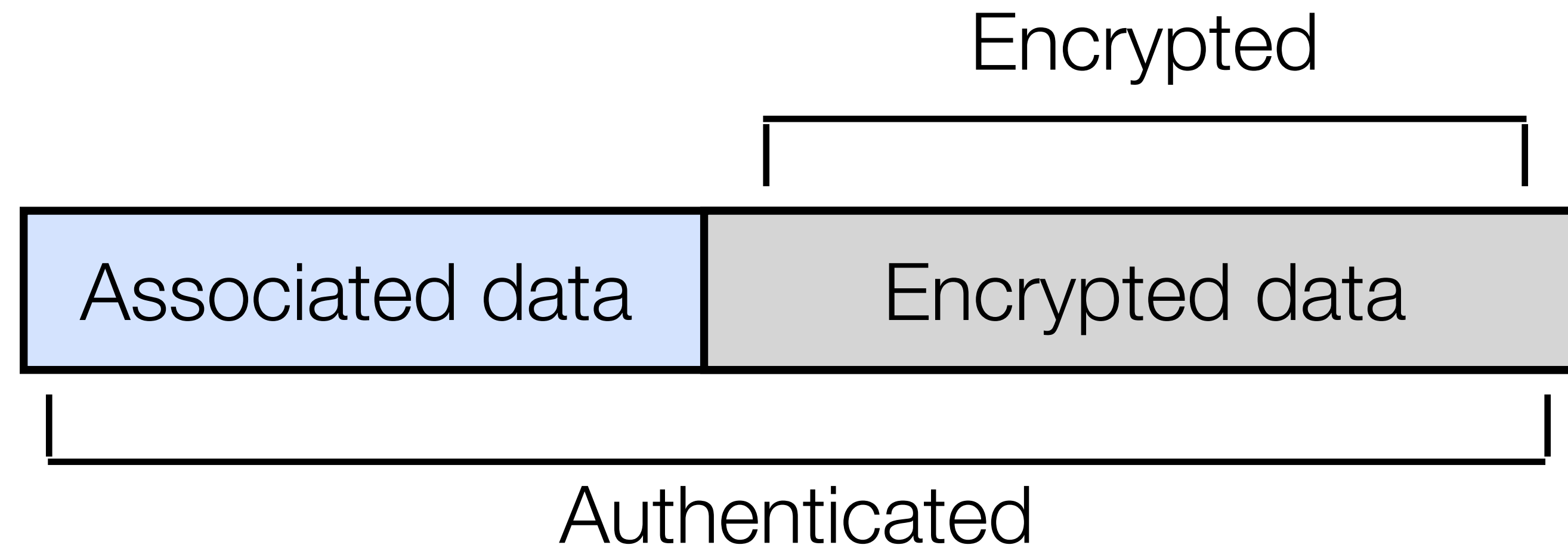
Always correct



Option 3: (SSH)



AEAD: Authenticated Encryption with Associated Data



AES-GCM: CTR-mode encryption then MAC

Public-key cryptography

Symmetric key vs. public key cryptography

Symmetric-key cryptography

Same key for:

- Encryption + decryption
- Signing + verifying MAC tags

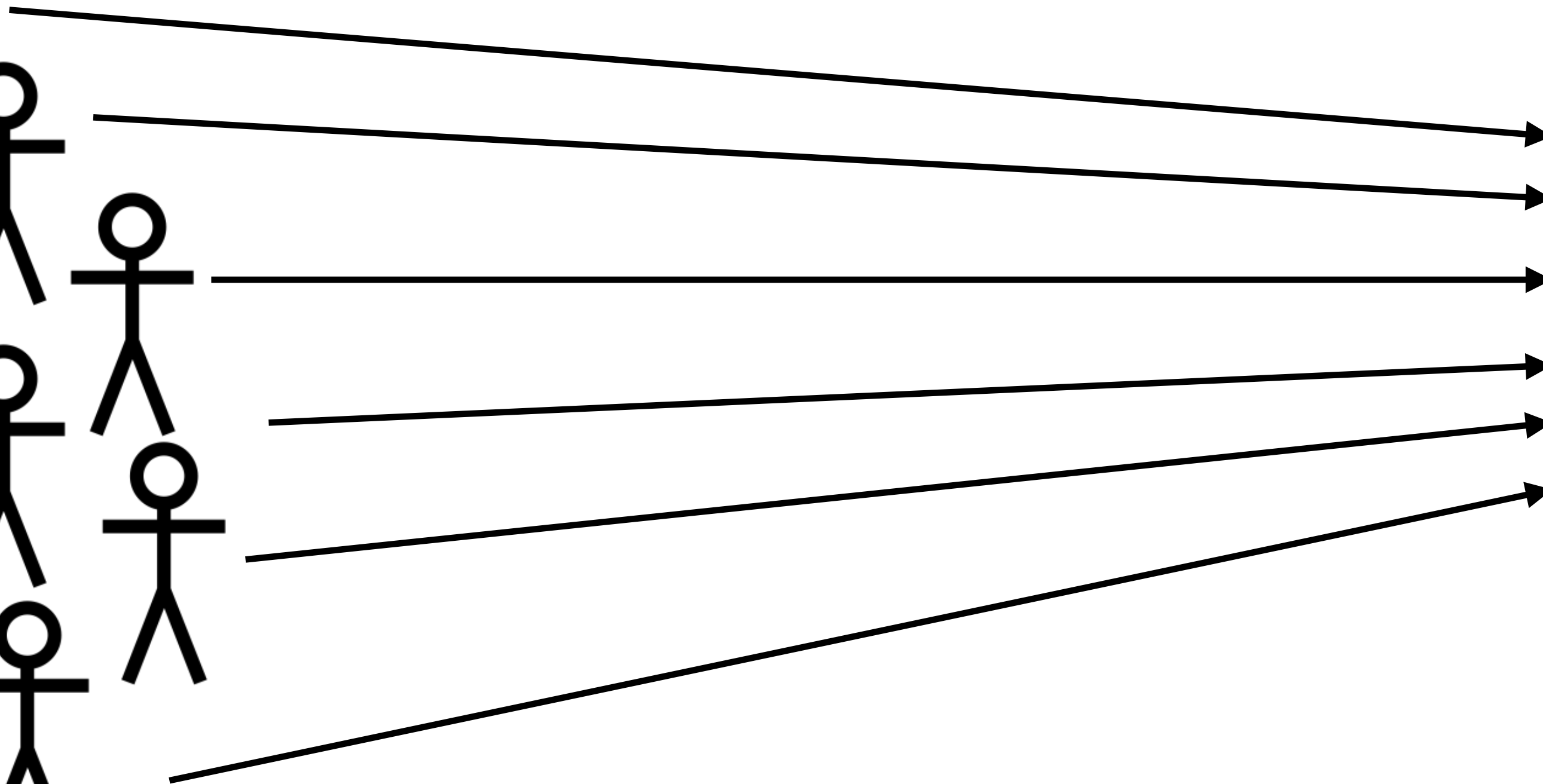
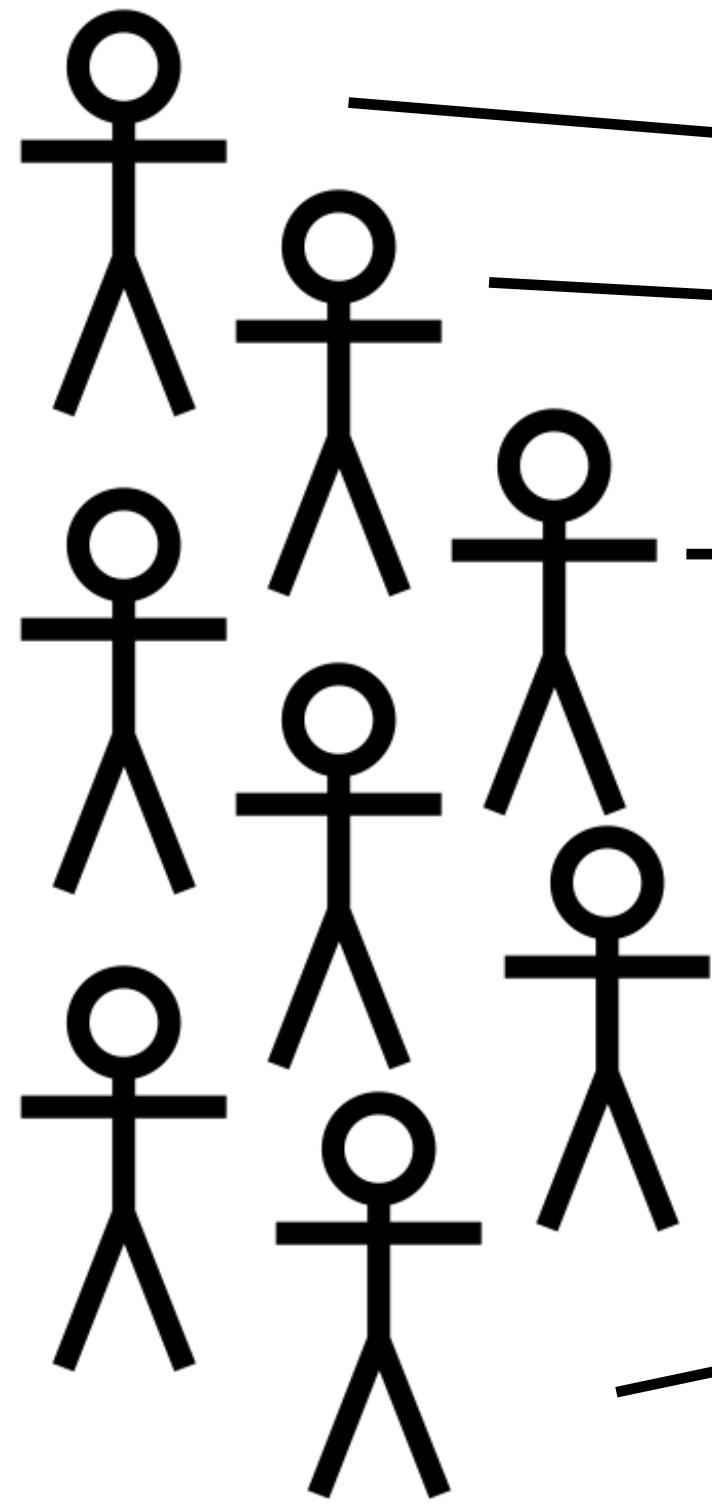
Public-key cryptography

Different keys for:

- Encryption + decryption
- Signing + verifying signatures

Public-key encryption

Public key pk



Secret key sk

Public-key encryption

$\text{KeyGen}() \rightarrow (\text{sk}, \text{pk})$

$\text{Enc}(\text{pk}, m) \rightarrow c$

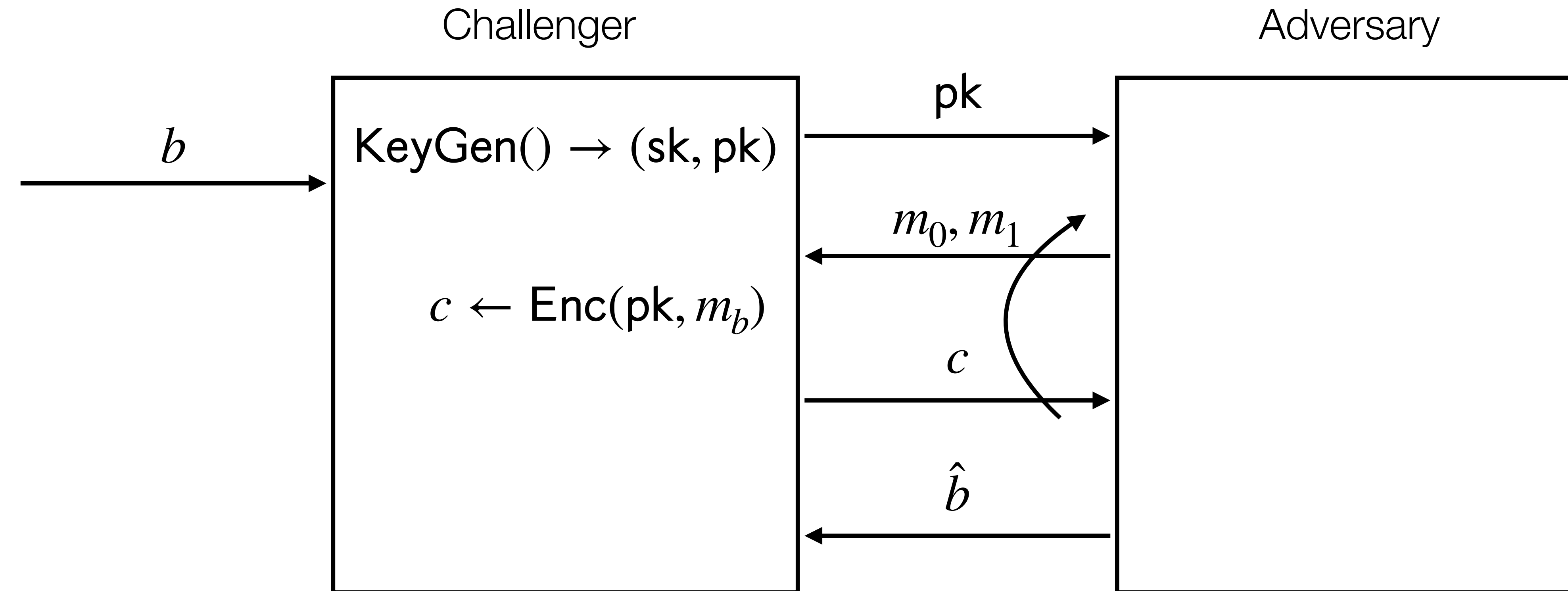
$\text{Dec}(\text{sk}, c) \rightarrow m$

Notes:

- Public key does not make it possible to decrypt
- Clients can generate encryptions without being able to decrypt
- More expensive than symmetric-key cryptography (but still fast enough to encrypt web traffic)

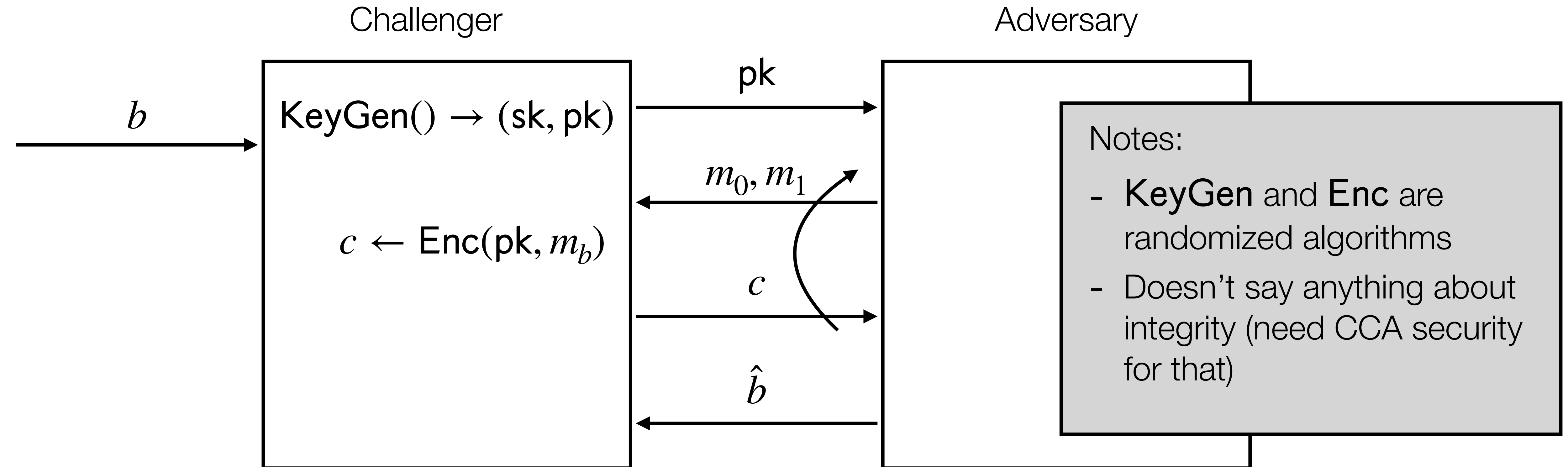
Examples: ElGamal, RSA, ...

Chosen plaintext attack security for public-key encryption



An encryption scheme is semantically secure if the adversary's probability of guessing $\hat{b} = b$ is "very close" to $1/2$.

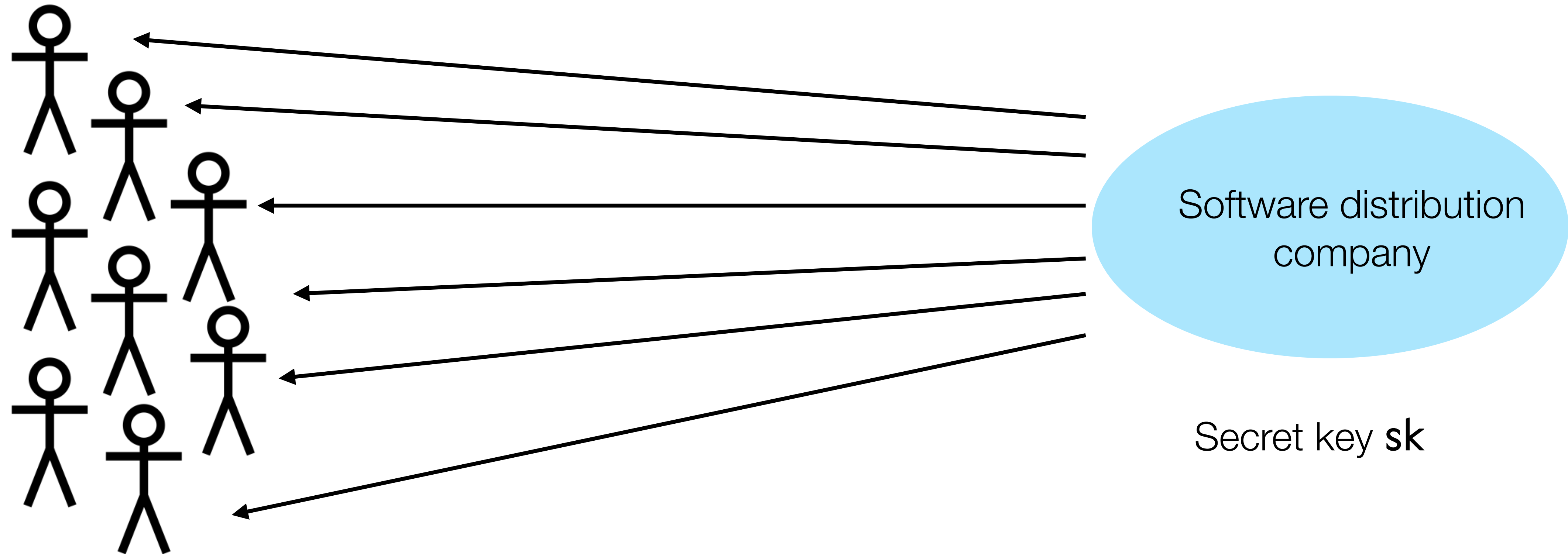
Chosen plaintext attack security for public-key encryption



An encryption scheme is semantically secure if the adversary's probability of guessing $\hat{b} = b$ is "very close" to $1/2$.

Digital signatures

Public key pk



Digital signatures

$\text{KeyGen}() \rightarrow (\text{sk}, \text{pk})$

$\text{Sign}(\text{sk}, m) \rightarrow \sigma$

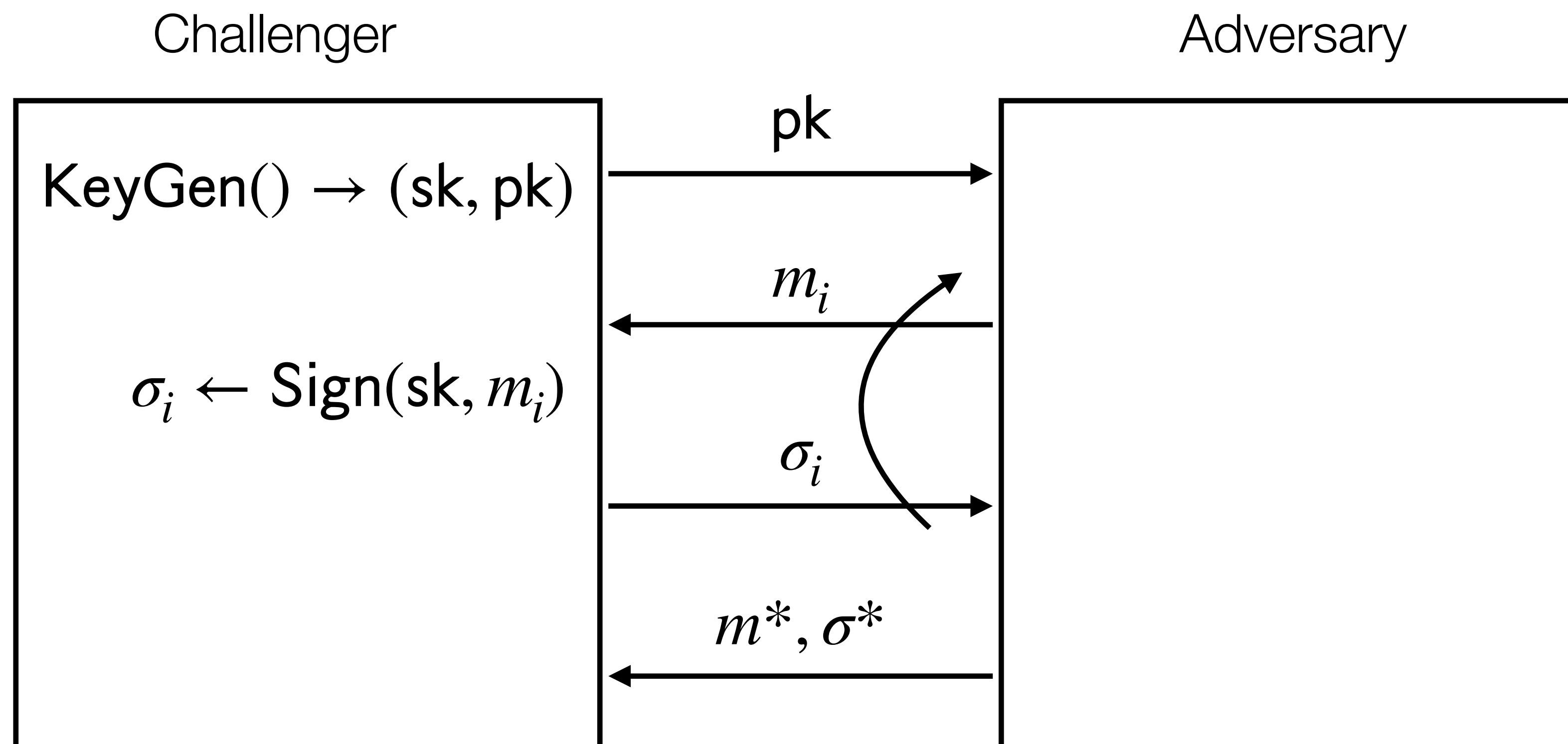
$\text{Verify}(\text{pk}, m, \sigma) \rightarrow \{0,1\}$

Notes:

- Public key only needed for verification
- Public-key analogue of message authentication codes

Examples: ECDSA, Schnorr, ...

Digital signature security



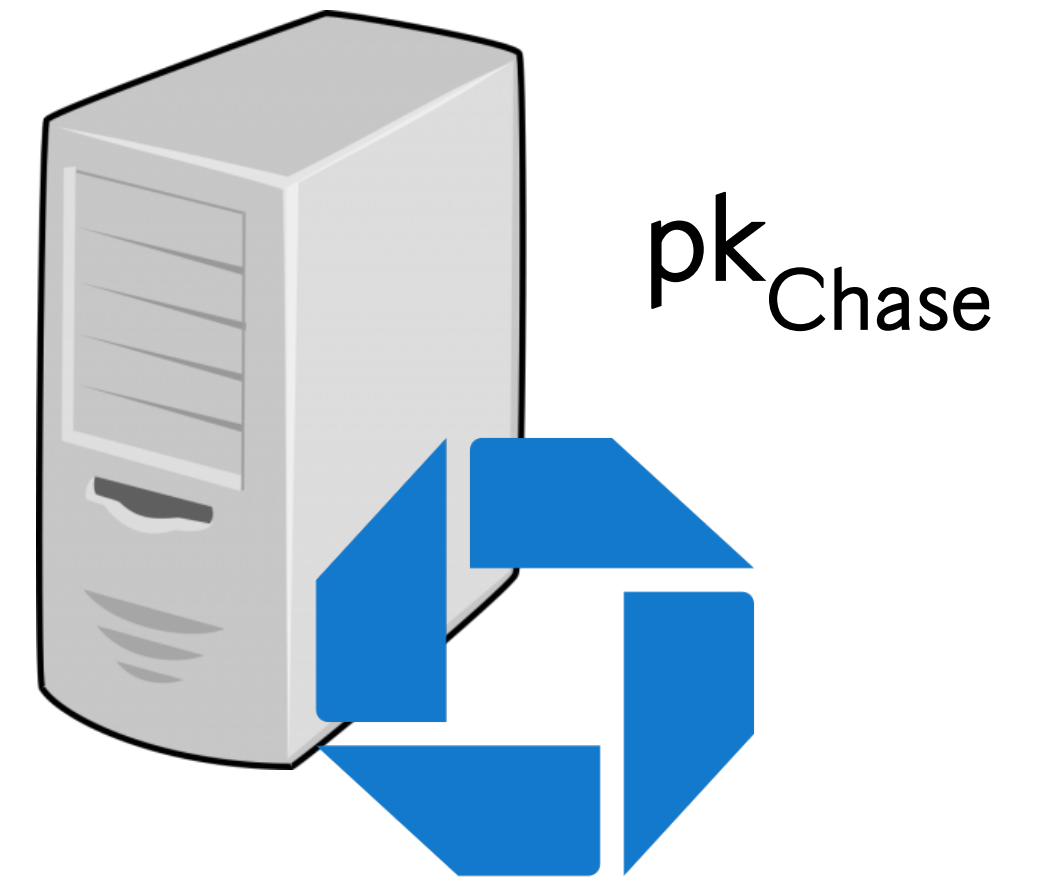
Adversary wins if:

- $(m^*, \sigma^*) \notin \{(m_1, \sigma_1), (m_2, \sigma_2), \dots\}$
- $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$ (“accept”)

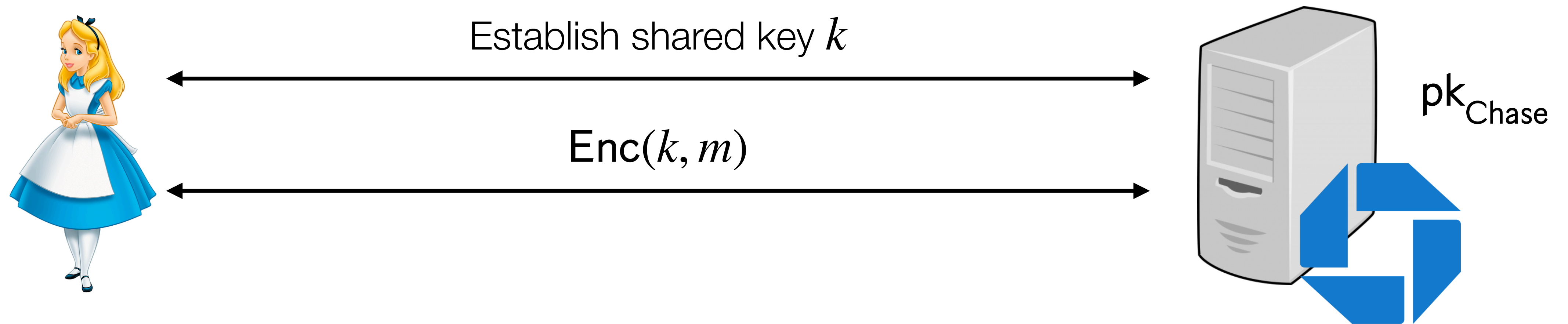
Stretch break

Browsing the web

How to securely establish a web connection?

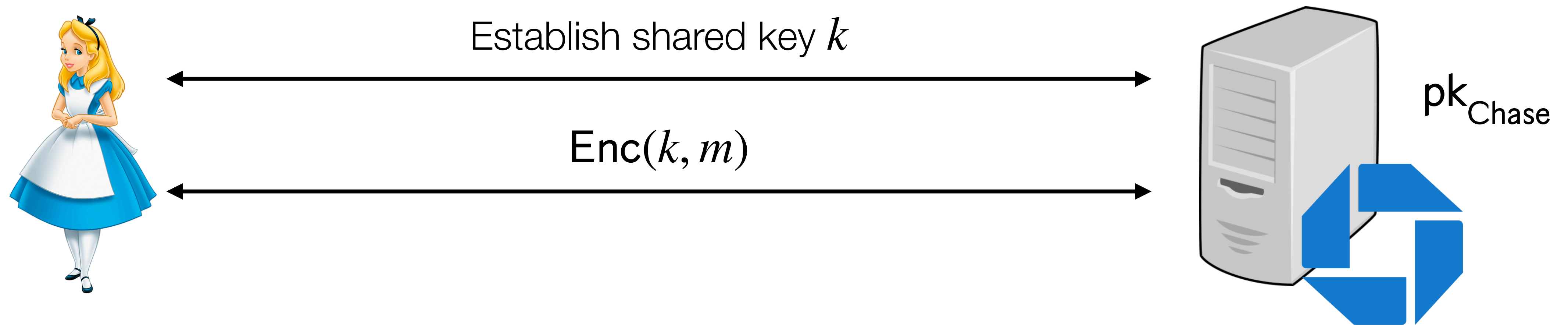


How to securely establish a web connection?



***Enc** is an authenticated encryption scheme

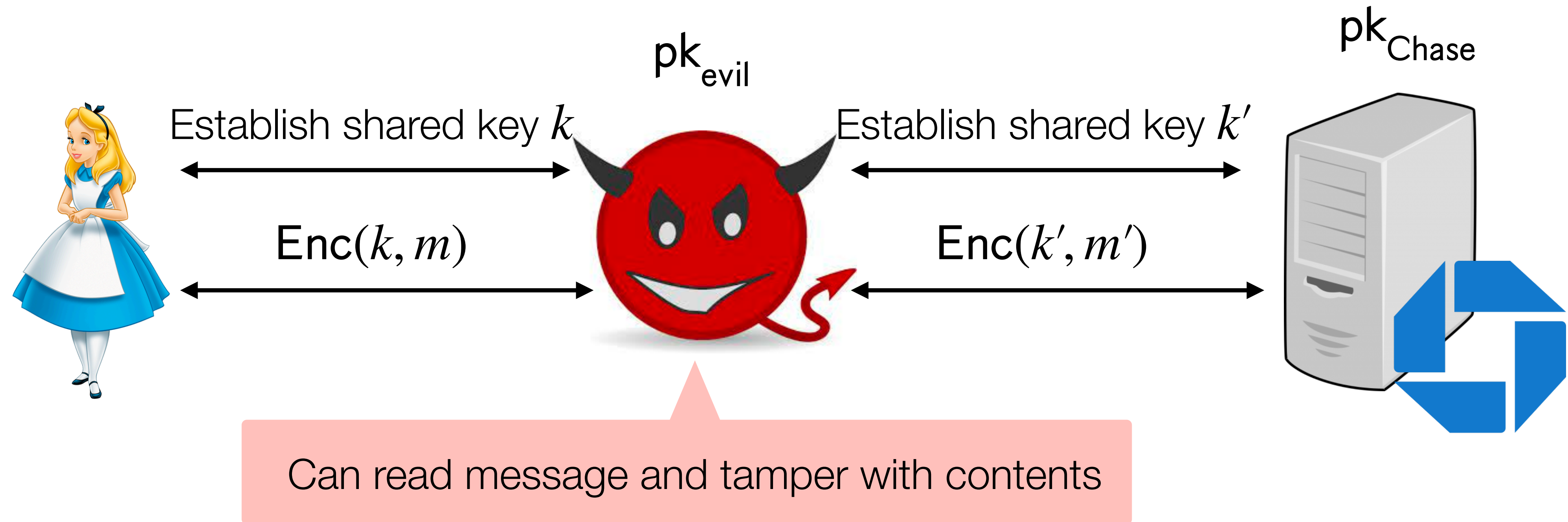
How to securely establish a web connection?



How does Alice know she is talking to the “right” pk_{Chase} ?

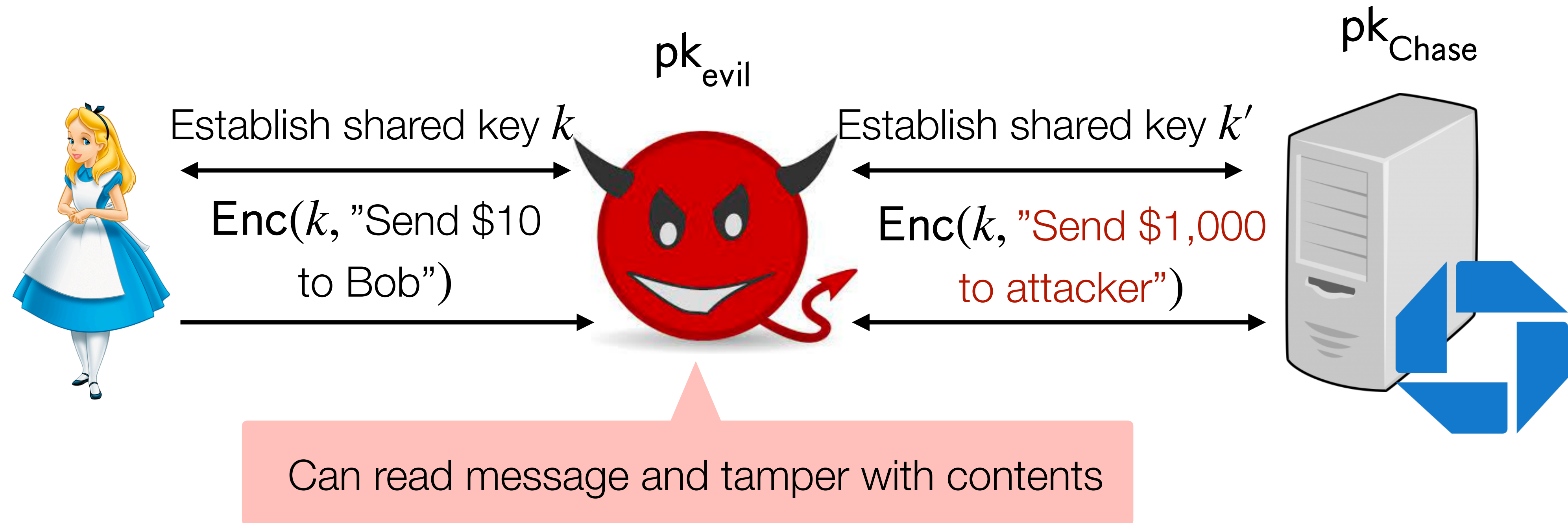
***Enc** is an authenticated encryption scheme

Man-in-the-middle attack



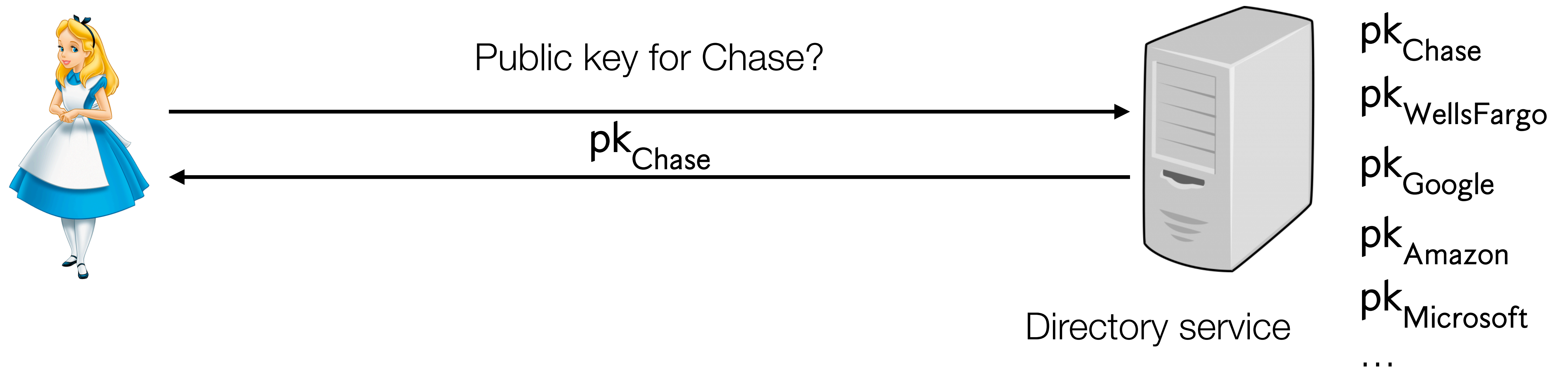
Alice cannot tell that she's talking to the attacker instead of Chase

Man-in-the-middle attack



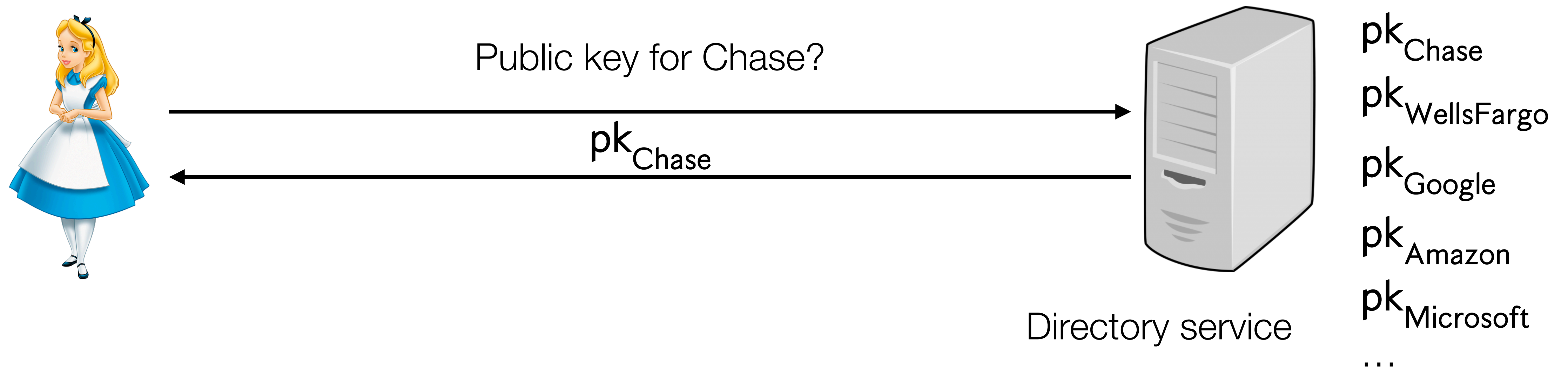
Alice cannot tell that she's talking to the attacker instead of Chase

One approach: trusted directory service



Drawbacks?

One approach: trusted directory service

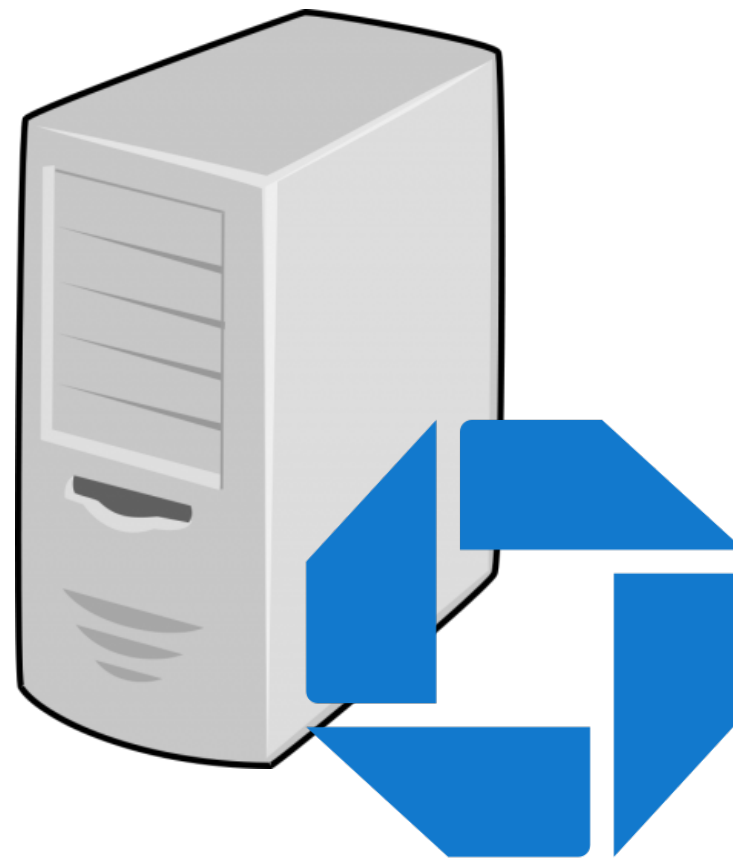


Drawbacks:

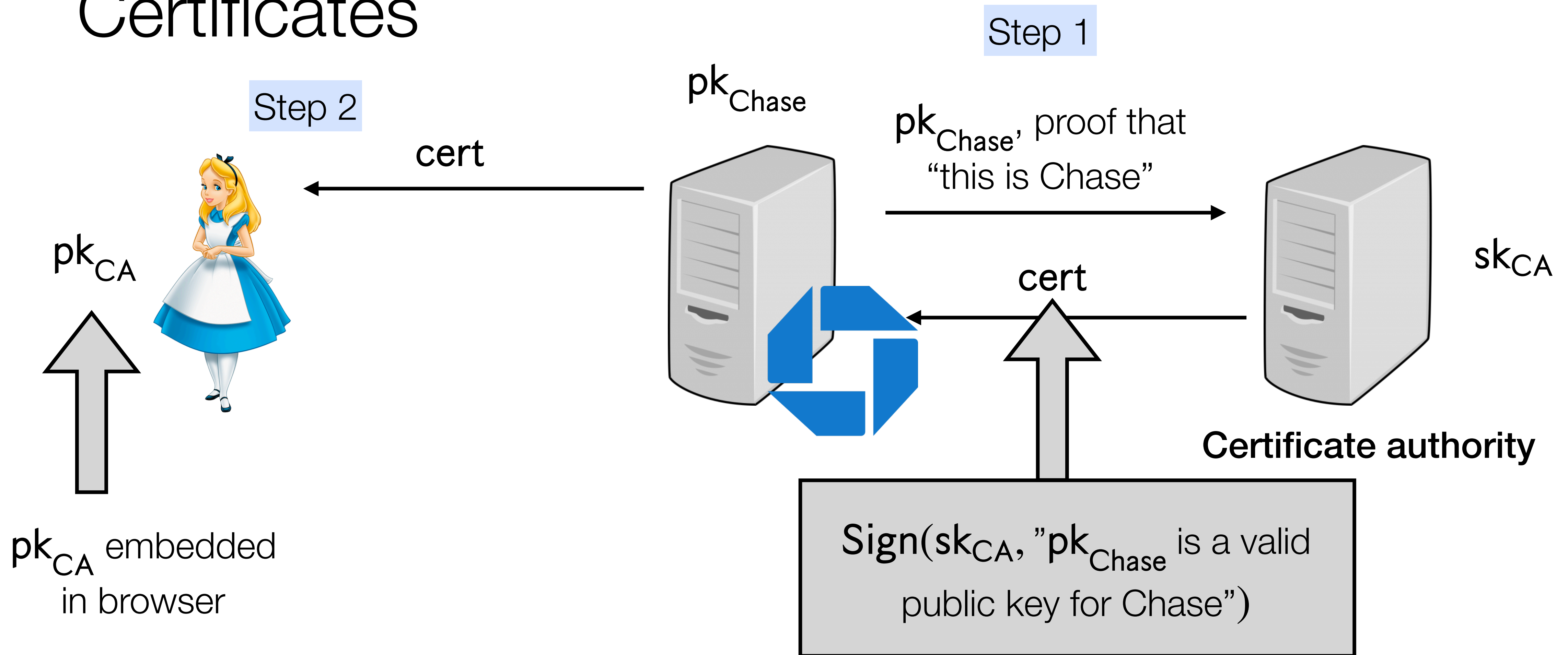
- A malicious directory service can provide the wrong public key
- Alice must be able to access the directory service at all times:
 - Must be online and can become a scalability bottleneck

The approach we use: certificates

pk_{Chase}

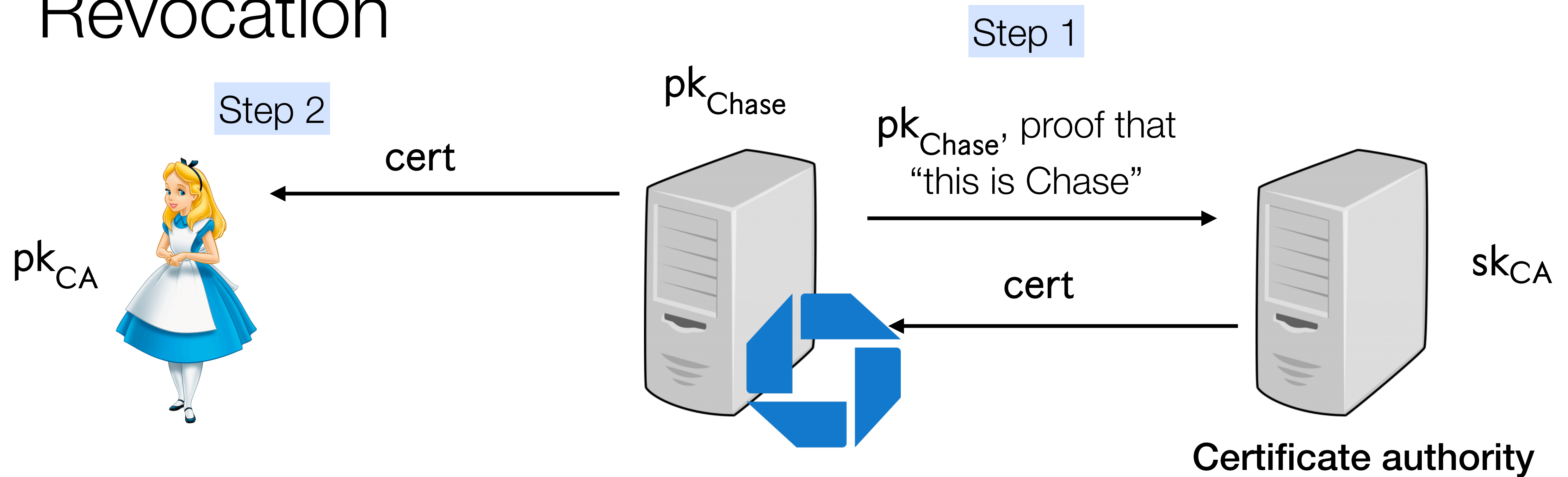


Certificates



Unlike directory service, Alice does not have to contact the CA

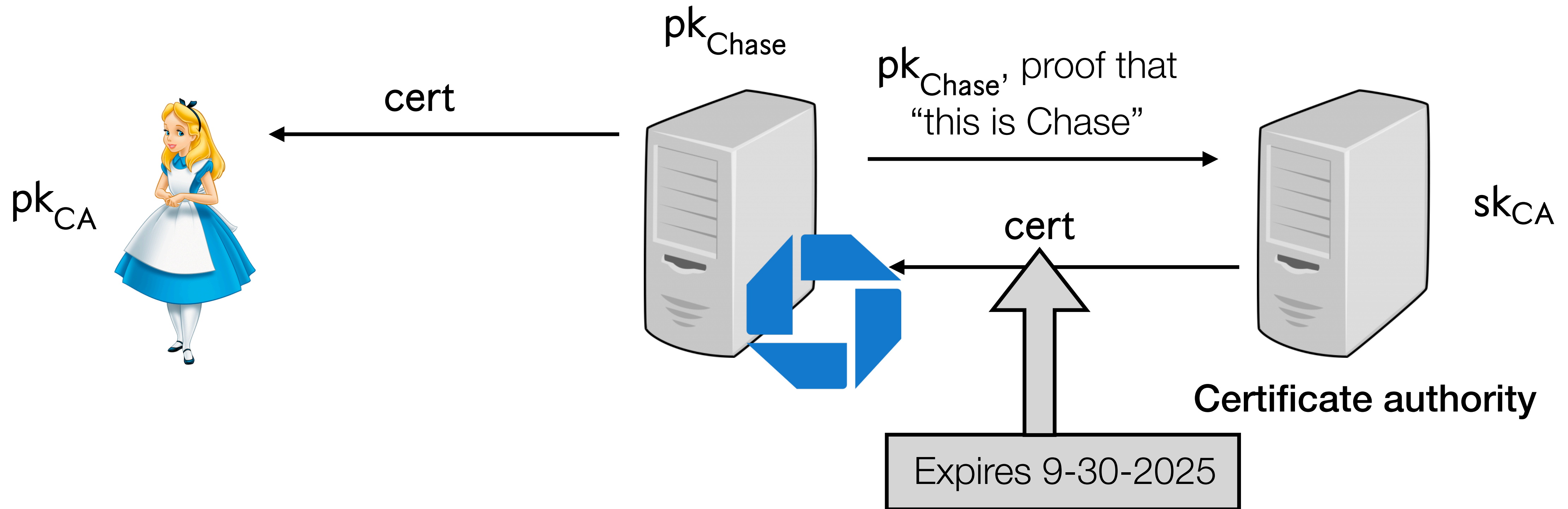
Revocation



The CA might make a mistake when issuing a certificate, or a website may need to update its public key

How to support updates with certificates?

Revocation approach #1: validity periods

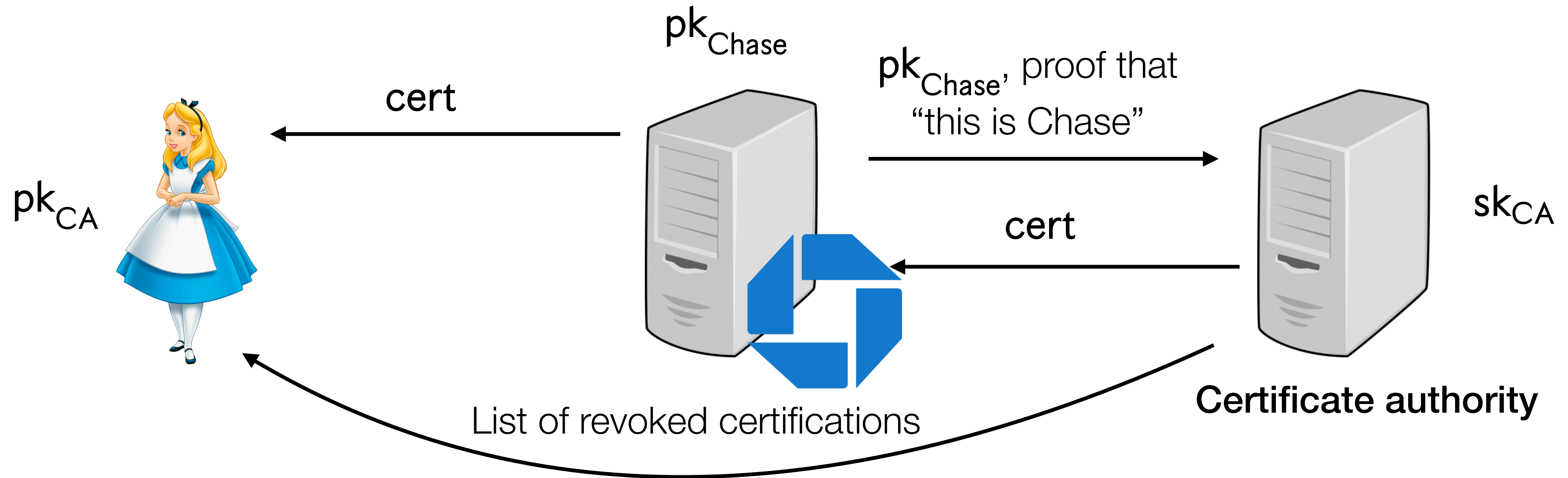


The CA might make a mistake when issuing a certificate, or a website may need to update its public key

How to support updates with certificates?

1. Validity periods: tradeoff between efficiency and speed of revocation

Revocation approach #2: revocation lists

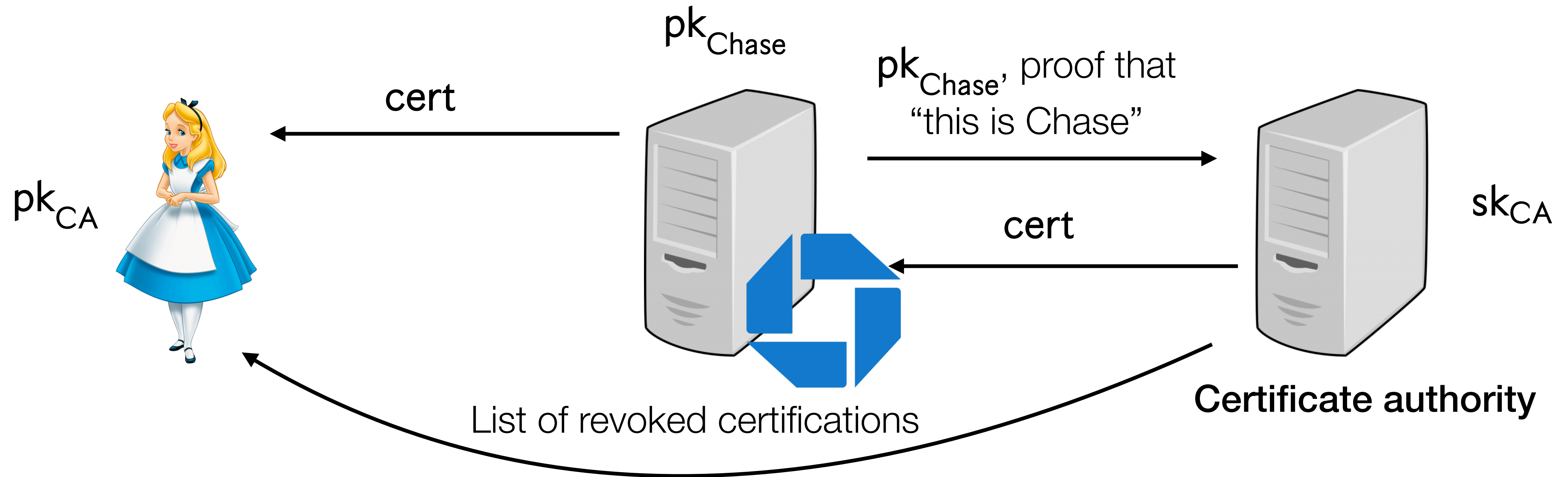


The CA might make a mistake when issuing a certificate, or a website may need to update its public key

How to support updates with certificates?

1. Validity periods: tradeoff between efficiency and speed of revocation
2. Revocation list: client periodically fetches list of revoked certificates

Revocation approach #2: revocation lists



Later class: how to protect against incorrectly issued certificates

Getting a certificate with Let's Encrypt

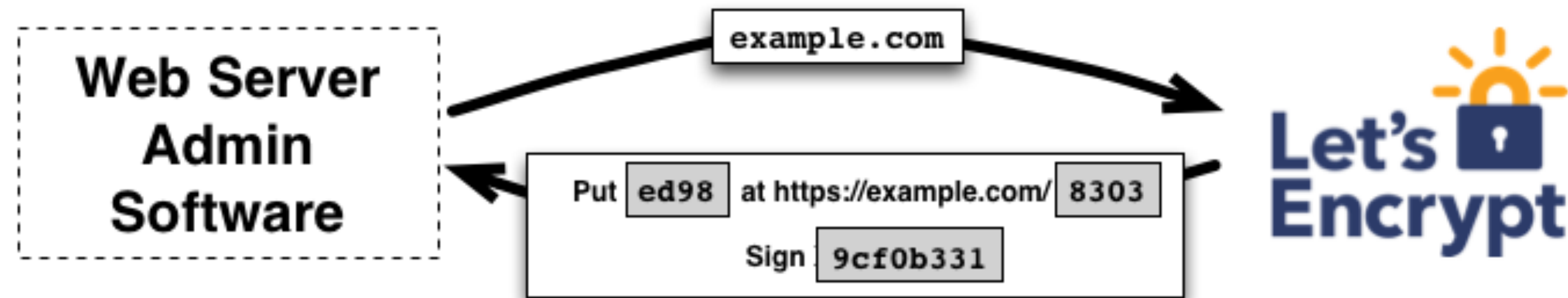


Free certificate authority that makes it easy to obtain certificates

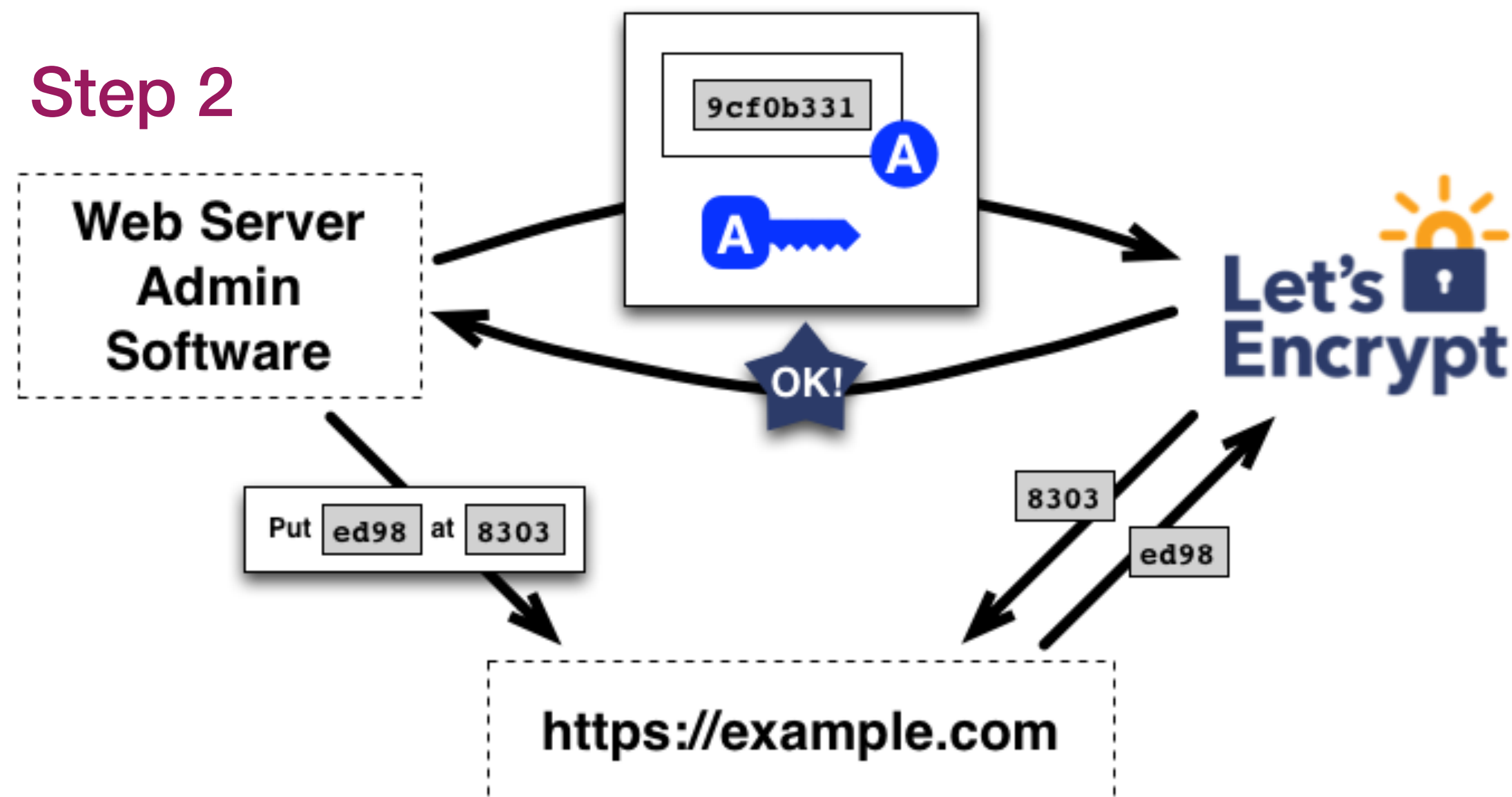
How to prove that you “own” a domain?

Getting a certificate with Let's Encrypt

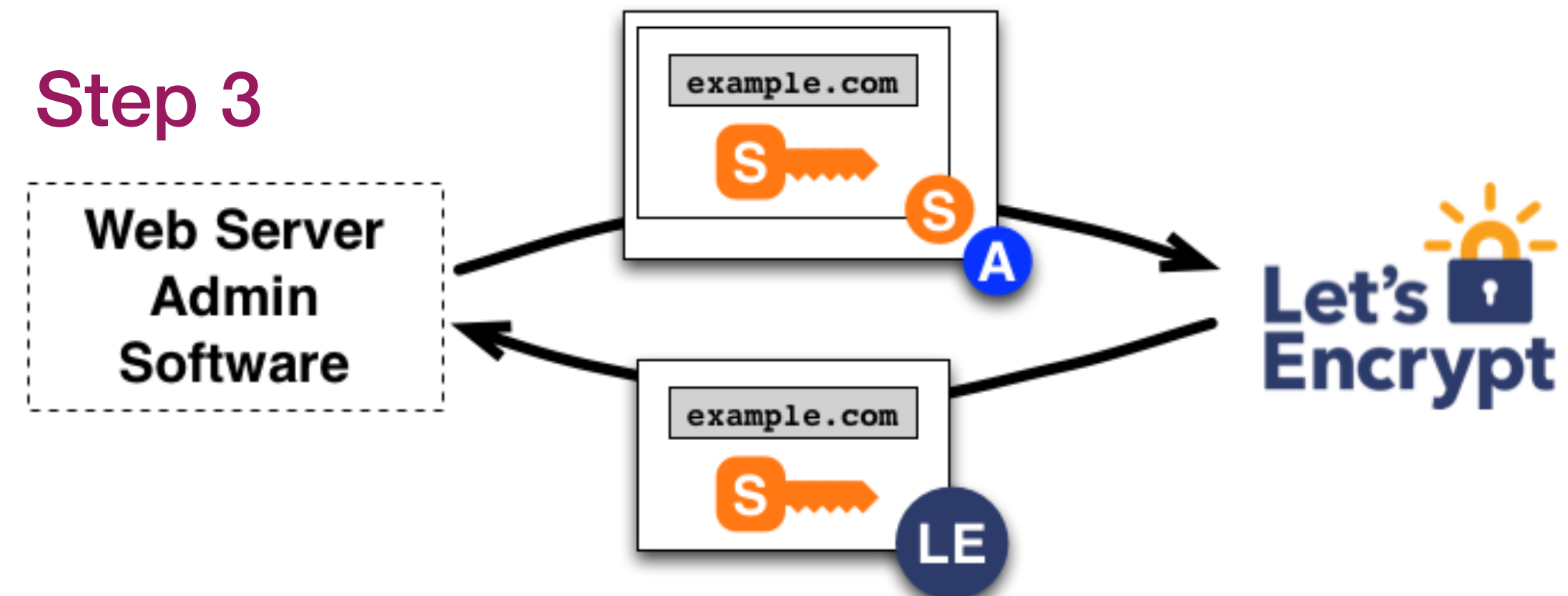
Step 1



Step 2



Step 3



Authenticated data structures

Goals for authenticated data structures

Limited storage



cm

Stores a short *commitment* to a list

Check if the commitment includes/excludes element v_i

Lots of storage



$v_1, v_2, v_3, \dots$

Stores the entire list

Tool: collision-resistant hash function

Input space $\{0,1\}^*$

Output space $\{0,1\}^{256}$

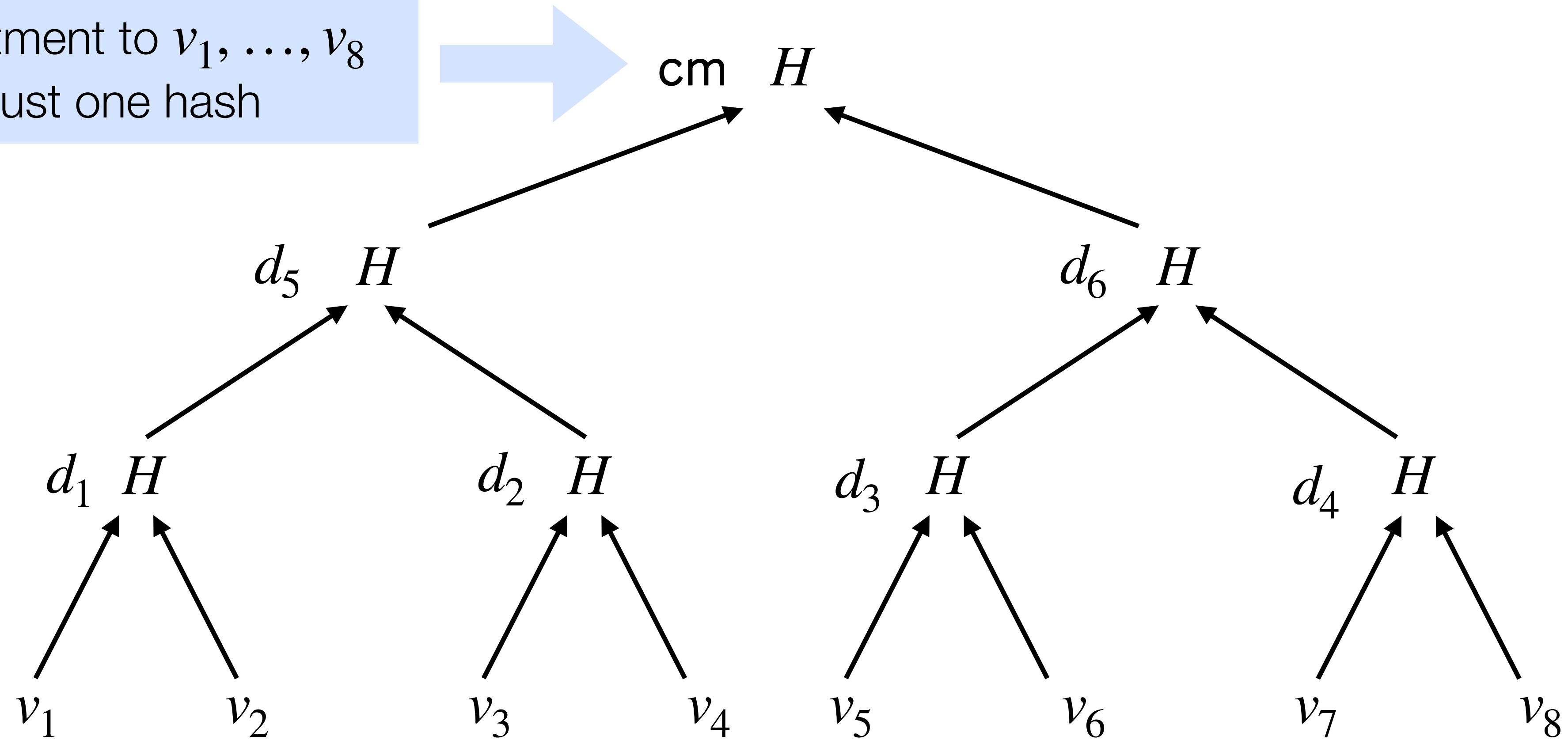
Hash function $H : \{0,1\}^* \rightarrow \{0,1\}^{256}$

A hash function H is collision-resistant if no “efficient” adversary can find a collision, i.e.

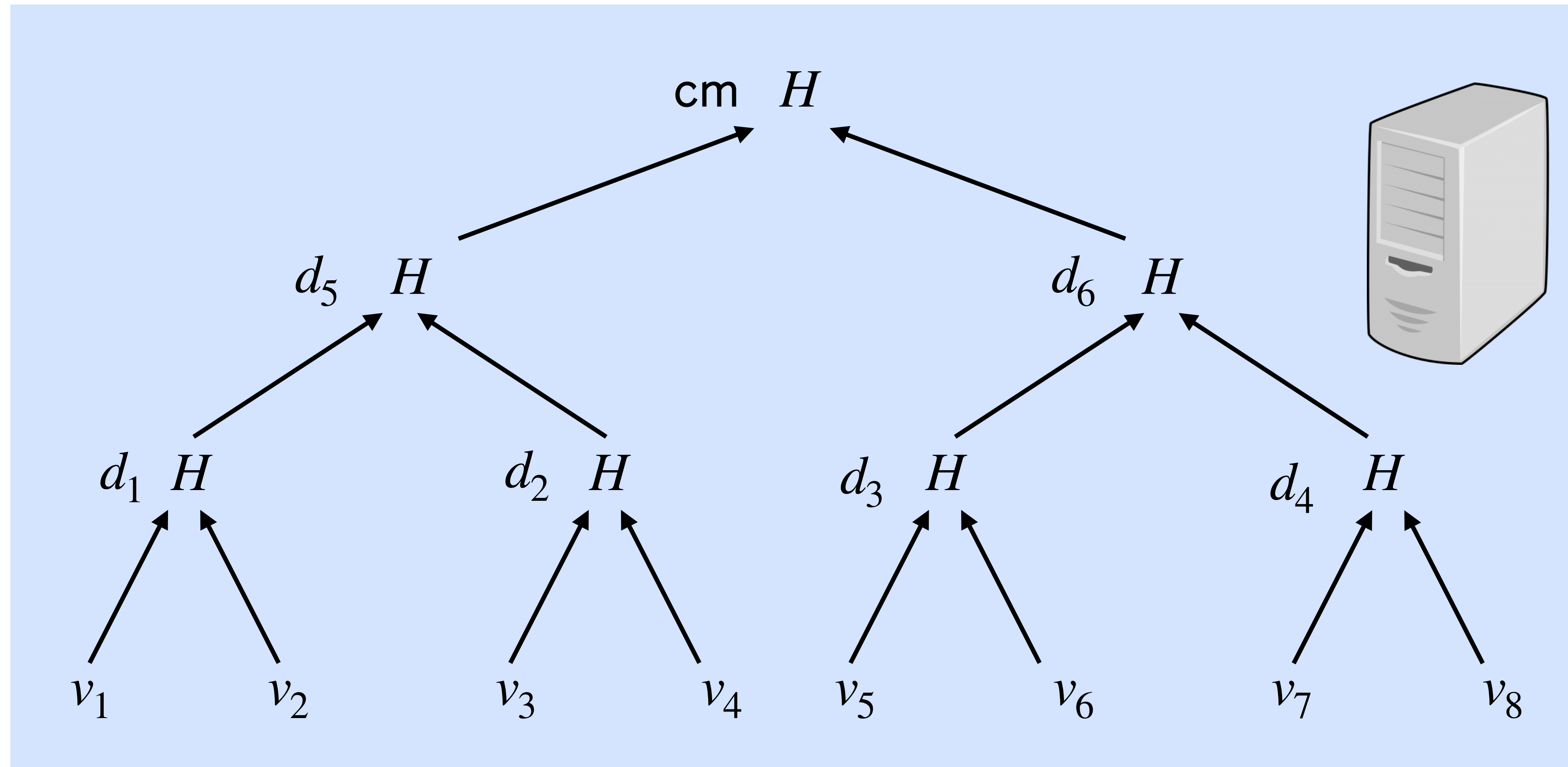
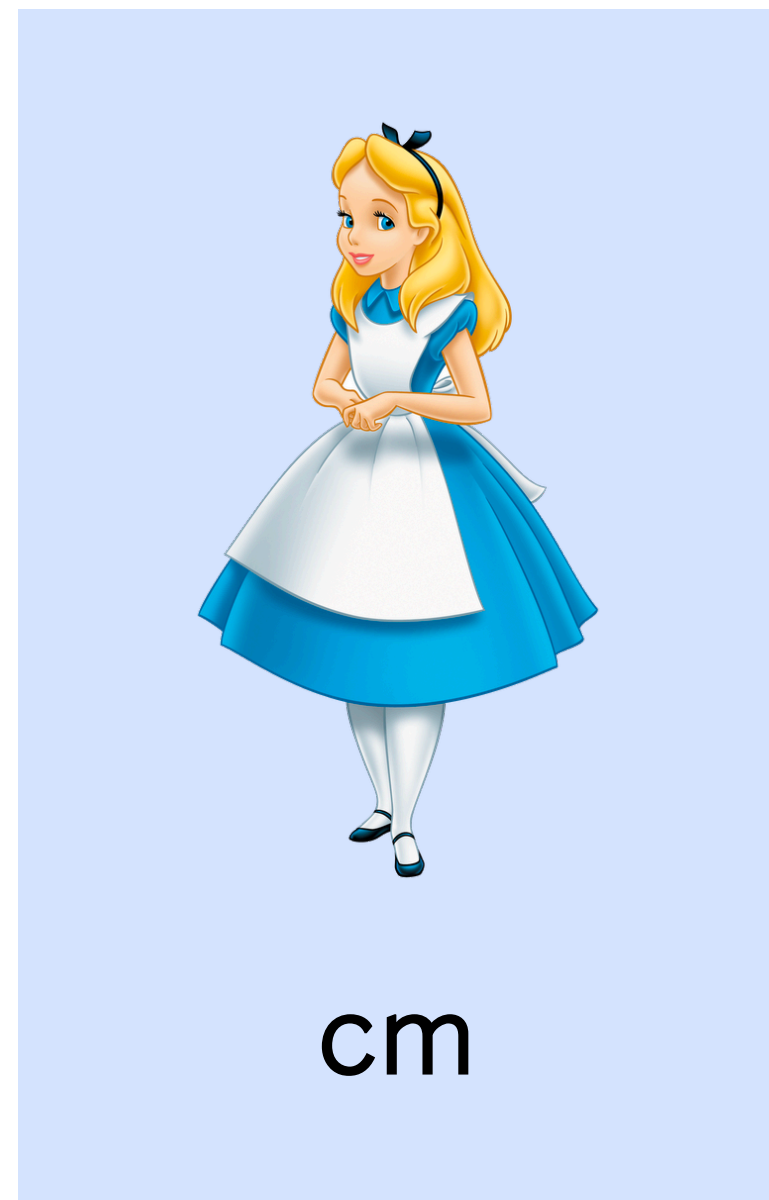
$H(a) = H(b)$ for $a \neq b$

Merkle tree

Commitment to $v_1, \dots, v_8$
is just one hash



Merkle tree



Merkle tree

Is v_3 included?

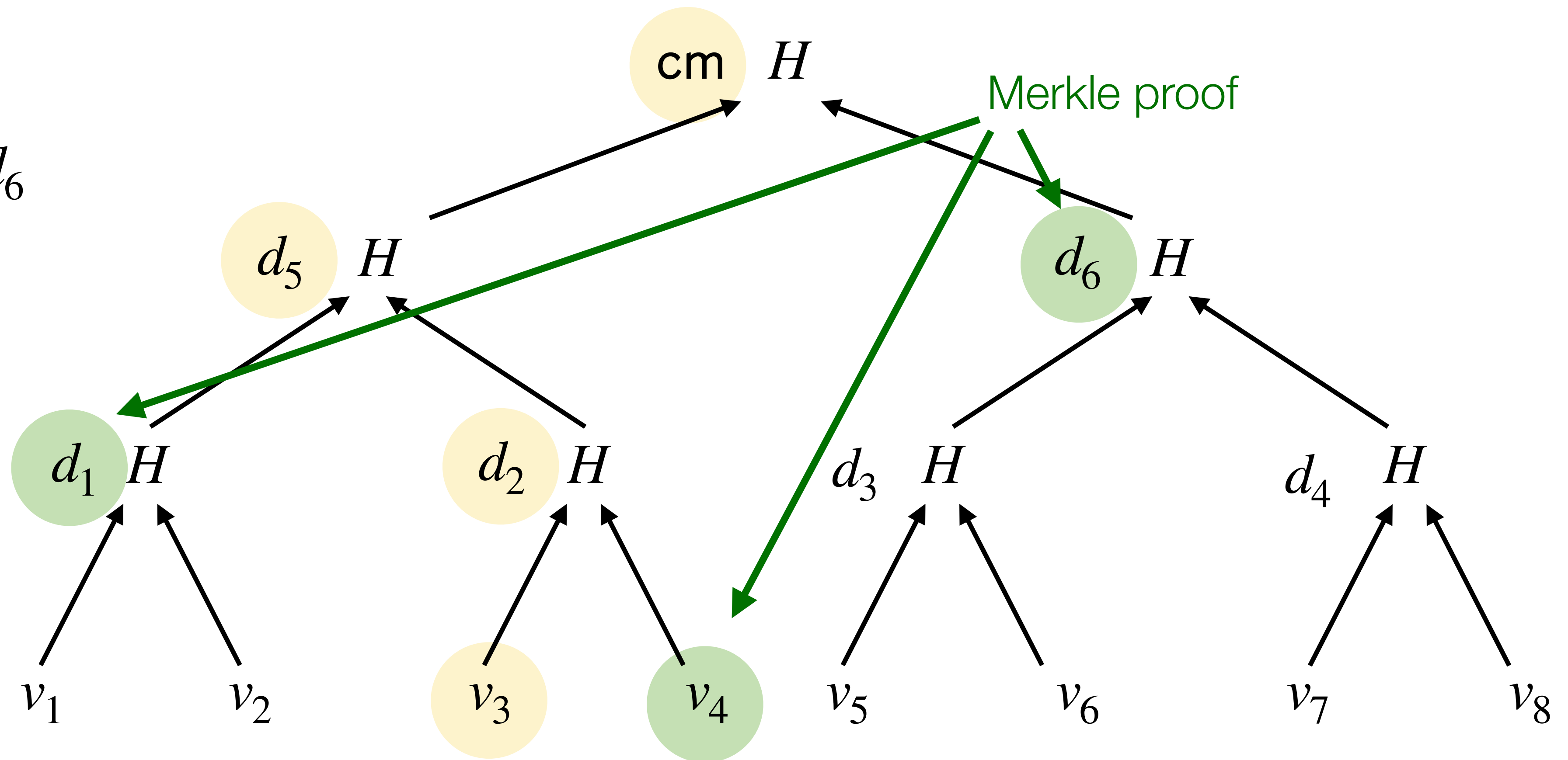
Fetch proof $\pi = v_4, d_1, d_6$

Compute:

- $\hat{d}_2 \leftarrow H(v_3, v_4)$

- $\hat{d}_5 \leftarrow H(d_1, \hat{d}_2)$

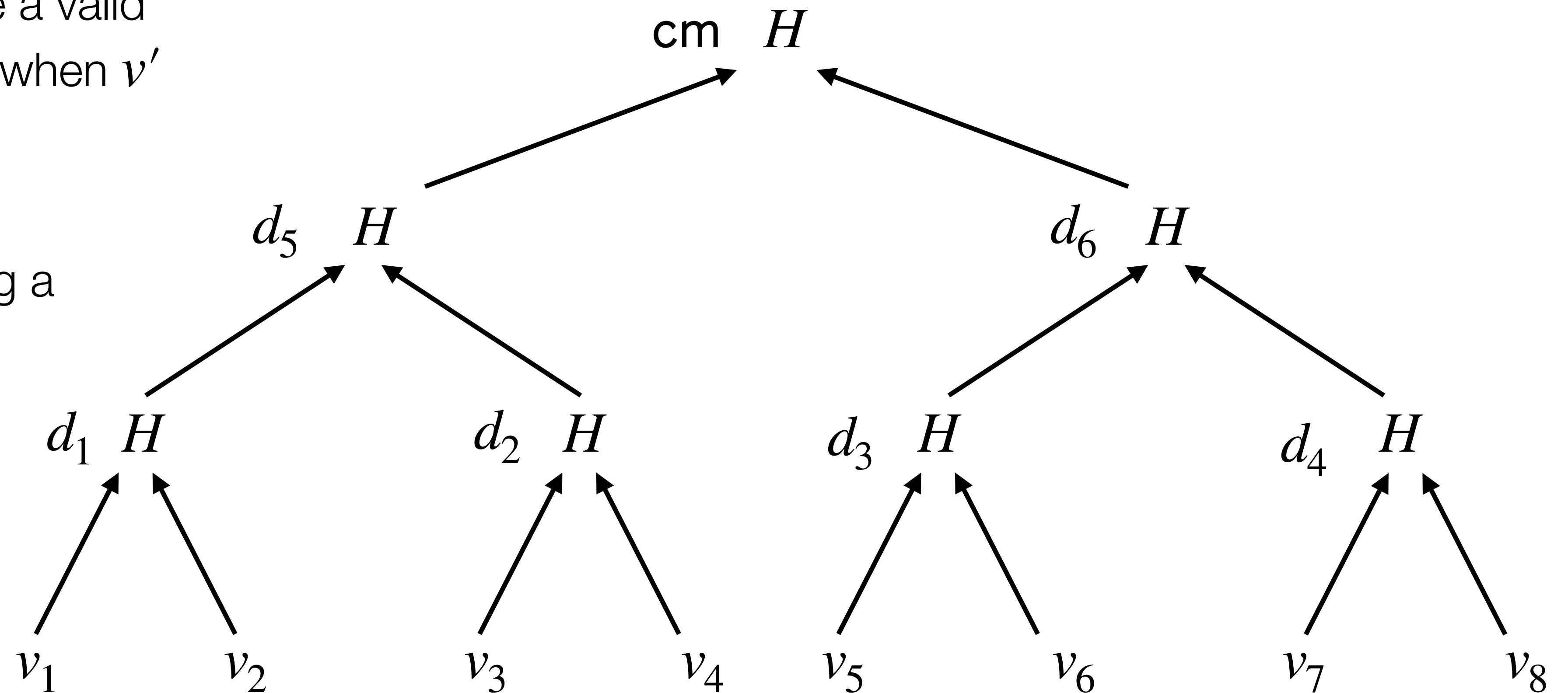
Check: $\text{cm} = H(\hat{d}_5, d_6)$



Merkle tree

Is it possible to generate a valid proof of inclusion for v' , when v' is not included?

No, would require finding a hash function collision

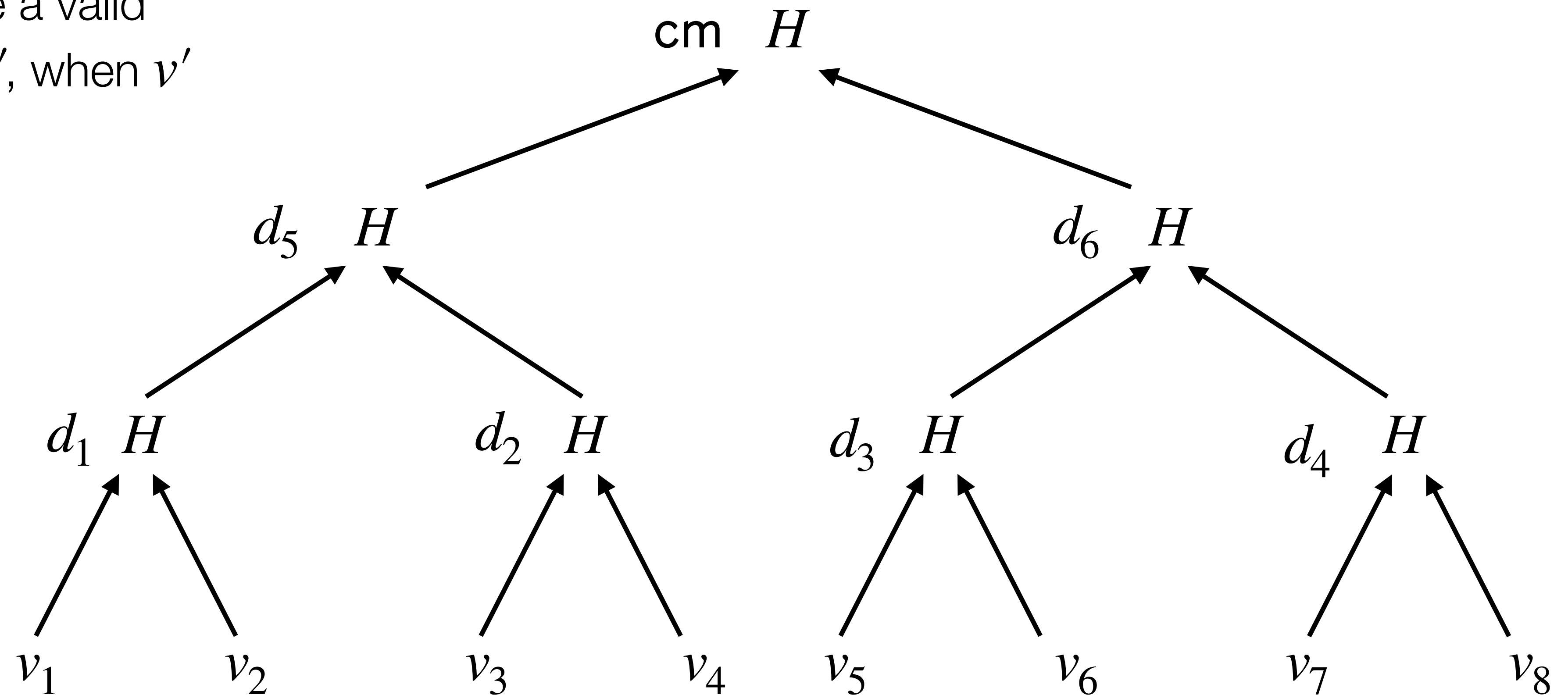


Authenticated data structures

Is it possible to generate a valid proof of **exclusion** for v' , when v' is not included?

Idea:

1. Sort the values
2. Generate proofs of the values to the right and left of where v' would go



Authenticated data structures

Sort values when creating Merkle tree

To generate proof of non-inclusion for v' relative to **cm**:

- Find adjacent v_i, v_{i+1} such that $v_i < v' < v_{i+1}$
- Generate Merkle proofs π_1, π_2 for v_i, v_{i+1}

To check proof of non-inclusion for v' :

- Check proofs π_1, π_2 relative to **cm**
- Check that $v_i < v' < v_{i+1}$ and v_i, v_{i+1} are adjacent

Requires list to be correctly sorted

Other applications of authenticated data structures

- Checking if files are correctly stored on a disk
- Blockchains
- Certificate transparency (this class)
- Key transparency (this class)

Next time: secure hardware

- Secure hardware classes:

1. Using secure hardware for analytics

Reading: Opaque

↳ First reading quiz!

2. Guest lecture: Sacha Servan-Schreiber (Tinfoil) on private AI

3. Using secure hardware without fully trusting it for encrypted backups

Reading: SafetyPin

- Fill out the form on Ed to sign up for paper presentations

End of lecture

References

Stanford CS 255, CS 251

MIT 6.1600

Berkeley CS 161

Boneh-Shoup cryptography book